

How to Keep Pushing ML Accelerator Performance? Know Your Rooflines!

Marian Verhelst¹, Fellow, IEEE, Luca Benini², Fellow, IEEE, and Naveen Verma³, Senior Member, IEEE

Abstract—The rapidly growing importance of machine learning (ML) applications, coupled with their ever-increasing model size and inference energy footprint, has created a strong need for specialized ML hardware architectures. Numerous ML accelerators have been explored and implemented, primarily to increase task-level throughput per unit area and reduce task-level energy consumption. This article surveys key trends toward these objectives for more efficient ML accelerators and provides a unifying framework to understand how compute and memory technologies/architectures interact to enhance system-level efficiency and performance. To achieve this, this article introduces an enhanced version of the roofline model and applies it to ML accelerators as an effective tool for understanding where various execution regimes fall within roofline bounds and how to maximize performance and efficiency under the roofline. Key concepts are illustrated with examples from state-of-the-art (SOTA) designs, with a view toward open research opportunities to further advance accelerator performance.

Index Terms—Chip design, energy efficiency, machine learning (ML) accelerators, memory hierarchy, processor architectures, quantization, roofline model, sparsity, throughput.

I. INTRODUCTION AND MOTIVATION

THE size of machine learning (ML) models has been growing rapidly across the last decade, as illustrated in Fig. 1. This growth led to an explosion in the required number of operations per ML training or inference tasks, as well as growing memory footprints. The growth rate in complexity of these models substantially outpaces Moore's law (Fig. 1). As a result, tracking the requirements in energy efficiency and throughput requires agility in design innovation and clarity around design tradeoffs.

This sustained innovation pressure has led to a myriad of dedicated hardware accelerators, exploiting specialized

efficiency-boosting techniques, tailored for the ML domain. This push toward specialized architectures is challenged by a growing diversity in workloads. The rapid evolution of ML models, in terms of their topologies (layer dimensions and connectivity), precision requirements, modalities and embedding/de-embedding approaches, etc., requires specialization yet without giving up on programmability, necessitating careful balance between flexibility and efficiency.

Industrial and academic research have responded to the challenging needs, achieving impressive nearly 100× performance improvement every 24 months, maintained over the past eight years, as shown in Fig. 1. Despite the wide variety of ML accelerator architectures in the neural processing units (NPUs) of commercial products and published in the literature, a large part of their performance improvements can, however, be attributed to different variants of a common list of effective optimizations, including: 1) maximizing parallelism; 2) exploiting (spatial and temporal) data reuse; 3) quantization; 4) sparsity; and 5) near- or in-memory compute. This article surveys these techniques and illustrates their key insights and tradeoffs with various examples from state-of-the-art (SOTA) designs.

As these techniques are deployed in widely diverging platforms and at different scales, from tiny/extreme-edge devices, to edge processors, to cloud systems, it is difficult to assess them relative to each other. We require a unifying framework to assess their impact and to gain a structured understanding of what sets efficiency and performance, as well as what the associated tradeoffs are. This article attempts to do this, leveraging and enhancing the roofline model for throughput and energy efficiency. The goal is to clearly visualize the impact of different architectural techniques through the roofline representation, thereby enhancing contextual/comparative understanding. Finally, this will lead to insights into fundamental tradeoff between different techniques impacting performance, ultimately leading to open questions and directions for future research.

This article is organized as follows. Section II outlines the components contributing to the energy and latency of ML accelerator execution, leading to a visualization comprising two different rooflines. The most impactful efficiency enhancement techniques from recent research are subsequently surveyed and assessed in light of the roofline models in Section III, illustrated by several examples from recent SOTA. Next, Section IV dives deeper into some fundamental tradeoffs between the presented techniques, and how they have a conflicting impact on the performance and efficiency of rooflines. This will finally lead to an outlook on the future, and directions for further research in Section V.

Received 7 November 2024; revised 17 February 2025; accepted 13 March 2025. Date of publication 21 April 2025; date of current version 29 May 2025. This article was approved by Associate Editor Dennis Sylvester. This work was supported in part by European Research Council (ERC); in part by the Flanders Artificial Intelligence (AI) Research (FAIR) Program; and in part by the Swiss State Secretariat for Education, Research, and Innovation (SERI) through the SwissChips Initiative and Defense Advanced Research Projects Agency (DARPA) Optimal Processing Technology Inside Memory Arrays (OPTIMA) under Grant HR00112490300. (Corresponding author: Marian Verhelst.)

Marian Verhelst is with the Department of Electrical Engineering, KU Leuven, 3001 Leuven, Belgium, and also with IMEC, 3001 Leuven, Belgium (e-mail: marian.verhelst@kuleuven.be).

Luca Benini is with D-ITET Department of Information Technology and Electrical Engineering, ETH Zurich, 8092 Zurich, Switzerland, and also with the DEI Department of Electrical, Electronic, And Information Engineering "Guglielmo Marconi", Università di Bologna, 40126 Bologna, Italy.

Naveen Verma is with the Department of Electrical and Computer Engineering, Princeton University, Princeton, NJ 08544 USA, and also with EnCharge AI, Santa Clara, CA 95054 USA.

Color versions of one or more figures in this article are available at <https://doi.org/10.1109/JSSC.2025.3553765>.

Digital Object Identifier 10.1109/JSSC.2025.3553765

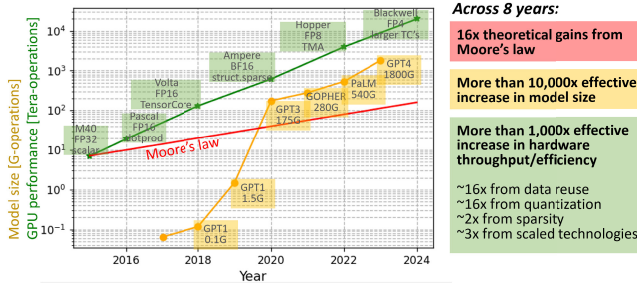


Fig. 1. Evolution of ML model size, GPU performance, and Moore's law across the last decade. Data from [1] and [2].

II. ROOFLINE MODELS FOR THROUGHPUT AND ENERGY EFFICIENCY

A. Where Does the Energy and Latency Go?

Almost all ML accelerators, regardless of whether standalone units, data-path extensions to a processor, or tensor cores integrate in a graphics processing unit (GPU), can be characterized by a common baseline architecture, as depicted in Fig. 2. A regular array of processing elements is each capable of one or more multiply-accumulate (MAC) operations. The data for these operations are fed from an on-chip memory system, typically consisting of a hierarchy of registers and one or more levels of static random access memory (SRAM) scratch pads. Additionally, the memory system typically couples to off-chip memory and/or storage subsystems.

To optimize ML accelerators for throughput and energy efficiency, one has to understand where the energy and latency are spent in such systems. While one could focus on the efficiency of the MAC operation itself, this often only contributes to a portion of the total system energy and latency. Specifically, the energy per inference task E_{task} and latency per inference task L_{task} are heavily impacted by the necessary data movement operations to bring the required data on chip, and to move the data between different on-chip memory levels.

For a given number of bytes N_{Li} that must be accessed in each level Li of the memory hierarchy in order to perform a given number of multiply and/or accumulate operations N_{op} per inference, the total energy per inference can be computed as

$$E_{\text{task}} = N_{op} \cdot E_{op} + \sum_i N_{Li} \cdot E_{Li} \quad (1)$$

with E_{Li} being the energy per byte of memory access at memory level Li , and E_{op} being the energy per multiply or accumulate operation.

Latency cannot be derived with a similar sum of the latency impact of different memory and compute components: as ML accelerators are typically designed to perform compute and data-movement operations in parallel, compute operations can be hidden behind memory transfers or vice versa. As a result, the total latency per inference task is typically obtained by

$$L_{\text{task}} = \frac{1}{f_{\text{clk}}} \cdot \max \left(\frac{N_{Ln}}{B_{Ln}}, \dots, \frac{N_{Li}}{B_{Li}}, \frac{N_{op}}{A_{op}} \right) \quad (2)$$

with B_{Li} being the bandwidth (bytes/cycle) of memory level Li , A_{op} being the amount of arithmetic operators physically on the chip, and f_{clk} being the chip's operating frequency.

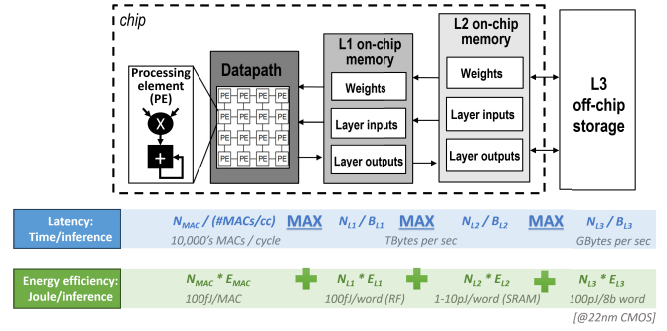


Fig. 2. Typical architecture of an ML accelerator. Note that the amount of PEs in the datapath, the number of memory levels in the memory hierarchy, and the type of memory at each level (registers, SRAM, and DRAM) can vary from system to system. Illustrative bandwidth and energy numbers are from a 22-nm technology.

To determine which contribution dominates the total energy and latency of an ML accelerator, one can fill in energy and memory bandwidth values of a target technology, as given in Fig. 2 for a 22-nm CMOS. Doing so, it quickly becomes apparent that in both the energy and latency models, the ratios between the operation counts N_{op} ¹ and different memory access counts N_{Li} strongly determine the relative contribution of different components toward system-level efficiency. In practice, these ratios are determined by the workload and denoted as the “arithmetic intensity” AI (also referred to as the “operational intensity,” or “compute intensity”) of the target workload for memory level Li [3]

$$AI_{Li} = \frac{N_{op}}{N_{Li}}. \quad (3)$$

For neural network workloads, the compute intensity typically increases steeply toward the upper levels of the memory stack. A single MAC can have an AI of less than one toward its internal registers, as maximally one operand can remain stationary [4] and thus multiple words have to be read/written per operation. Yet, higher up in the memory hierarchy, the compute intensity can be drastically higher due to various data reuse opportunities, as will be discussed further in Section III.

It is important to note that the achievable AIs toward different memory levels are highly network and layer-dependent [5]. While CONV operators with large input/output tensors might be able to achieve high AIs in the upper level memories, others such as fully connected layers in batch-1 mode experience low AIs, which are rather flat along the memory hierarchy. For this reason, it is important to assess performance as a function of AI, which will be discussed in Section II-B.

B. Not One, But Two Roofline Models

To assess throughput across a wide range of potential AIs, the computer architecture community introduced the “roofline model” [3], [6]. This model was later also applied to large language model (LLM) inference on GPUs [7]. It provides a visualization of the attainable processor performance as a function of AI, to quickly assess which system components

¹Throughout the complete article, only addition and multiply operations are considered toward the number of operations (ops), where a MAC counts for two operations.

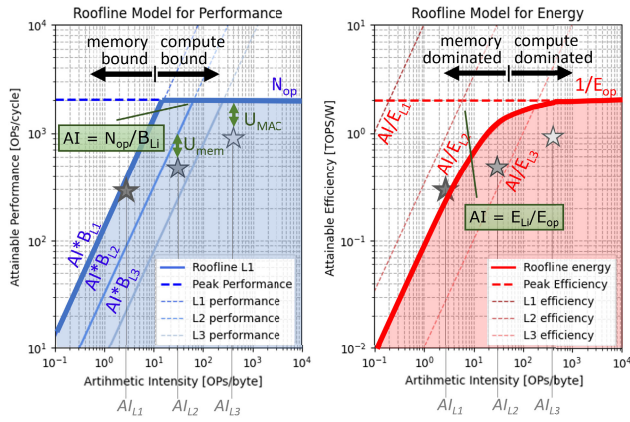


Fig. 3. Example roofline models for performance and energy efficiency (assumptions: $N_{op} = 2048$, $E_{op} = 0.50$ pJ/op, $E_{L1} = 0.100$ pJ/B, $E_{L2} = 3$ pJ/B, $E_{L3} = 100$ pJ/B, $B_{L1} = 128$ B/cycle, $B_{L2} = 32$ B/cycle, $B_{L3} = 8$ B/cycle, and for the aggregated energy roofline: $AI = AI_{L3}/16 = AI_{L2} = AI_{L1} * 16$).

limit performance for a given workload. The roofline models presented earlier, however, typically focus on the throughput performance of a processor with a single memory level, assuming the AI of the studied workloads remains almost identical across the memory hierarchy, rendering the highest level memory interface the most limiting one. An important distinction in ML accelerators is that the AI of neural networks typically varies widely between memory levels. The opportunities to multi-cast input and weight data, and reduce (aggregate) output data typically cause $N_{L3} < N_{L2} < N_{L1}$ and thus $AI_{L3} > AI_{L2} > AI_{L1}$ (see also Section III-B and [8]). This requires us to derive the roofline equation for total attainable throughput performance P_{TP} (expressed in operations/s), starting from (2), as

$$P_{TP} = \frac{N_{op}}{L_{task}} = \frac{1}{f_{clk}} \cdot \min(AI_{Ln} \cdot B_{Ln}, \dots, AI_{L1} \cdot B_{L1}, A_{op}). \quad (4)$$

Visualizing this results in Fig. 3 (left), in which the memory architecture sets the sloped performance boundary for lower AIs, while the MAC compute architecture sets the flat performance boundary for higher AIs. Depending on the relative memory bandwidths and AIs of different memory levels, one memory level will be the limiting one and form the overall system roofline [e.g., Level 1 in Fig. 3 (left)].

While not as common in the literature, it is also valuable to define an *energy* performance roofline [9] for ML accelerators. Energy efficiency is, however, not computed with a min/max operator, but as the sum of the contributing system components [see (1)], resulting in the following expression for the equivalent energy performance P_E , expressed in ops/s/W or ops/J²:

$$P_E = \frac{N_{op}}{E_{task}} = \frac{1}{E_{op} + \sum_i \frac{E_{Li}}{AI_{Li}}}. \quad (5)$$

Visualizing this roofline will no longer give the typical roof shape seen in the well-known throughput curve, yet it will

result in a curved roofline, with a low-AI region where memory energy dominates the total efficiency, a high-AI region when compute energy dominates; and a middle region, where the curve is smooth because both components comparably impact total efficiency. The location of the bending point depends on the relative ratio of different (E_{Li}/AI_{Li}) terms for the memory levels Li .

It is important to note that the cutoff point of both rooflines can lie at very different AIs. While the performance roofline has its transition from the memory-bound regime to the compute-bound regime at $AI = N_{op}/B_{Li}$, the energy roofline curves at $AI = E_{Li}/E_{op}$ for a given memory level Li . As both depend on very different parameters, an accelerator can operate for a certain range of AIs simultaneously in the compute-bound regime for throughput performance and in the memory-bound regime for energy efficiency. As a result, optimizations can differ significantly depending on whether one cares more for throughput or energy efficiency. Finally, note that the throughput roofline only has a sharp transition as long as the compute and memory latency can overlap [min-operation in (4)]. In case total latency depends on a linear combination of both, also here a rounded roofline curve would appear similar to the energy roofline, as observed in benchmarks in [10].

While these roofline plots help us in determining the maximum attainable throughput and energy efficiencies for a given AI, neither allow us to derive the *effective* efficiency when executing a realistic workload. The actual performance will fall below the roofline curve, the extent of which depends on the effective *memory and compute utilizations*, U_{mem} and U_{MAC} , respectively. The performance in the flat part of the roofline will be lowered by the compute utilization U_{MAC} , characterizing the ratio of the average number of useful MAC operations performed per cycle to the total amount of MAC operations available in the ML accelerator. The performance in the sloped part of the roofline, on the other hand, will be lowered by the memory utilization U_{mem} , representing the ratio of the average number of useful data bytes accessed from a memory level per cycle, to the peak memory bandwidth at that level.

C. Optimizing Performance Under the Rooflines

The goal of ML accelerator designers, system architects, and workload mappers is to maximize throughput and energy performance. From the discussions in Section II-B, it is clear that this can be done through: 1) raising the rooflines themselves; 2) by moving to the right, to the compute-bound regime; and/or 3) by more closely approaching the rooflines (Fig. 4). Here, we briefly summarize the most relevant SOTA methods toward these three approaches and visualize their effect in Fig. 4. We will dive into more detail on each of them in Section III.

1) *Raising the Rooflines*: On the one hand, increasing the compute boundary peak performance (operations per cycle, N_{op}) and peak MAC efficiency (operations/s/W, $1/E_{op}$) allows us to raise the flat part of the roofline. Fig. 4 outlines various architectural techniques (parallelization for large compute arrays and multi-processors, analog compute, etc.), as well

²In the remainder of this article, we will consistently express energy efficiency in terms of TOPS/W = 10^{12} ops/s/W, which is equivalent to ops/pJ

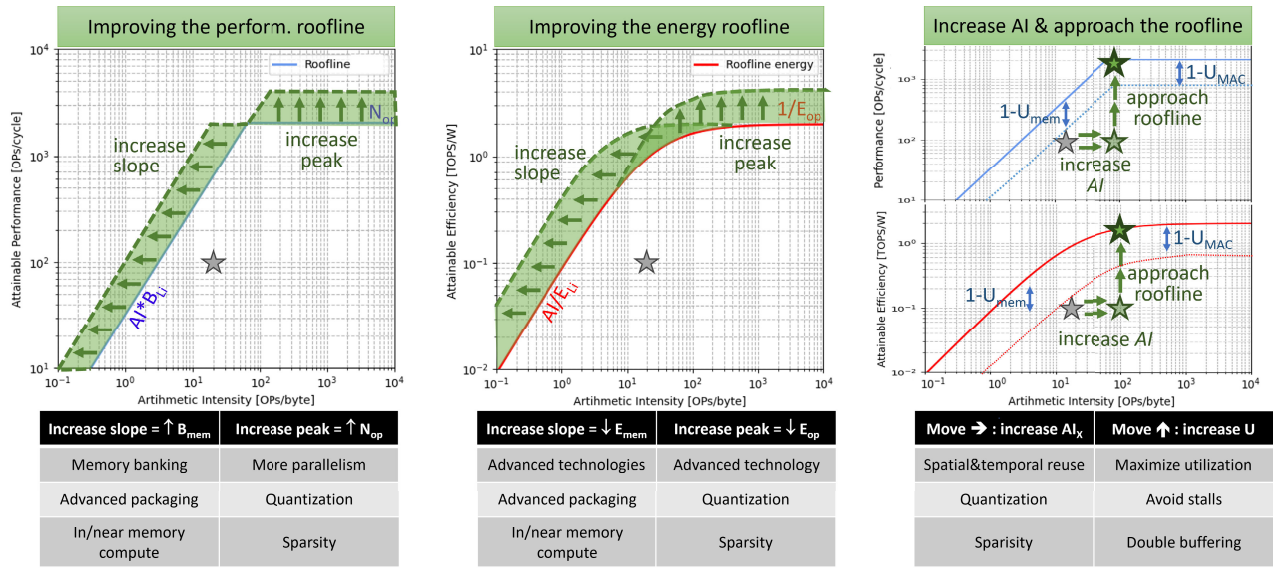


Fig. 4. Overview of various architectural and mapping techniques to maximize performance by raising the rooflines and more closely approaching the rooflines.

as algorithmic techniques (quantization, sparsity, etc.). On the other hand, increasing the memory boundary in terms of memory bandwidth (bits/cycle, B_{mem}) and memory efficiency (energy per access, $1/E_{mem}$) allows us to raise the diagonal part of the roofline and move the knee point more to the left, i.e., to lower AI. This, for instance, is possible by optimizing memory architectures for more efficient parallel access, as well as through near/in-memory compute.

2) *Move to the Compute-Bound Regime:* As long as workloads operate on the left of the roofline knee point, compute performance and energy efficiency are bounded by the memory bandwidth of the system. To maximally exploit a systems hardware resources, operation should take place as close to the knee point as possible. This typically requires to move right on the horizontal axis, shifting to larger AI or in other words doing more compute per fetched data element. One technique to increase AI, is to perform more aggressive quantization, as reducing the precision of the operands increases the number of operations per data byte. However, also the hardware architecture can influence the AI. While AI is often considered to only be a function of the considered workload and not of the hardware architecture, we will show in this article that this is not the case for NPUs. The physical compute architecture can enable a larger AI by, for example, providing larger on-chip caches or hardware support for spatial and temporal data reuse, or by supporting quantization, and/or sparsity exploitation.

3) *Approaching the Roofline:* Under a fixed throughput or energy performance roofline and for a given AI, effective performance can still be situated well below the roofline. Both in the memory-bounded and in the compute-bounded regions, effective performance can be limited by utilization. For memory-bounded workloads, this stems from the fact that the memory bandwidth is not used to the fullest. For compute-bounded workloads, this stems from idle cycles or idle MACs on the compute array. In order to maximize performance, these utilization losses should be eliminated to move as close as possible to the roofline. This requires minimizing all efficiency

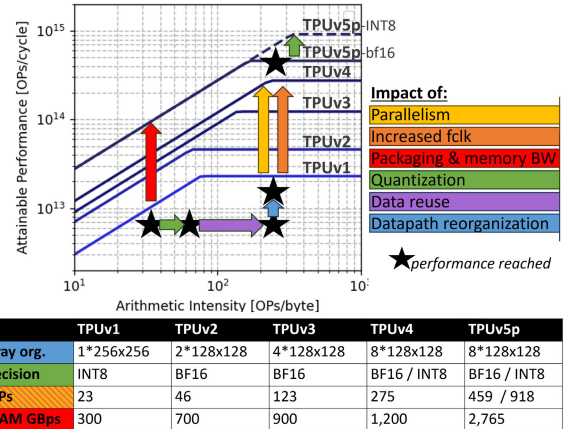


Fig. 5. Evolution of the rooflines of different TPU generations, together with the impact on AI of a typical GeMM workload. Data from [11].

losses stemming from memory and compute under-utilization, by, for example, foreseeing support for double buffering to avoid memory stalls or datapath reconfigurability to maximize utilization.

Before we dive deeper into those, let us look at how different techniques come together in a recent NPU evolution. Fig. 5 shows the performance of subsequent generations of Google's TPU. As can be seen, different techniques, such as a wider the dynamic random access memory (DRAM) bandwidth, increased parallelism, and a higher clock frequency, are used to raise the roofline (1). Second, hardware support for quantization and data reuse is added to move the workloads to the right of the AI axis (2). For example, the TPU makes use of large 128×128 compute arrays for massive spatial data reuse and is equipped with support for both BF16 and INT8 as of the v4 generation. Finally, hardware is modified to pursue a high utilization in order to operate close to the roofline curve (3). Specifically, the array size is brought from 256×256 to 128×128 to reduce compute utilization losses, while all weight registers are double buffered to reduce memory stalls.

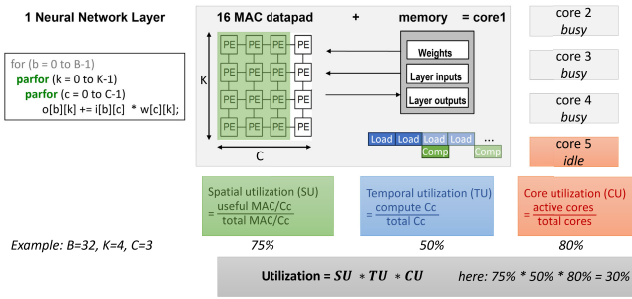


Fig. 6. Spatial and temporal under-utilization. Illustration for a 5-core system with an (unrealistically small) MAC array of dimension 4×4 per core, and a MatMul workload of dimensions $32 \times 4 \times 3$.

In the combination of these different techniques, it is important to always keep an eye on the X -axis location of the throughput and energy knee points. Depending on the AI range of the target workload, one or the other might be more relevant. While improvements of the AI and utilization have an impact on the performance under both rooflines, several other architectural enhancements only impact one of the two rooflines. Examples are changes in parallelization (N_{op}) shifting the performance roofline, versus changes in operation or memory energy (E_{op} or E_{Li}) only impacting the energy roofline. After this bird's eye view, let us dive deeper into each of these techniques, to fully understand the opportunities that stem from each of them in light of the performance under the roofline.

III. HARDWARE TECHNIQUES TO MAXIMIZE PERFORMANCE UNDER THE ROOFLINE

This section will dive deeper into various hardware architectural and implementation techniques aiming to increase throughput and/or energy efficiency.

A. Maximizing Parallelism

A direct knob to push the performance roofline up is to increase the amount of parallelism in the ML accelerator. The majority of the neural network operators consist mainly of MAC operations. To raise the flat part of the roofline, the number of parallel MAC units in the processing core should, hence, be increased. Indeed, this trend can clearly be observed over the past decade: from Eyeriss' 168 and Envision's 256 MAC units in 2016 and 2017 [4], [12], over Samsung's and Tesla's ~ 10000 of MACs in 2019 and 2020 [13], [14], to today's massively parallel near- and in-memory compute engines with 100 000s of parallel MAC operators [15], [16], [17], [18], [19], and often beyond this in the case of multi-core compute engines for providing additional parallelism at the full system level.

Rising to these levels of parallelism, however, comes with important challenges. First, a mapper is required to determine how to spatially and temporally map a target workload across the many parallel processing elements. Assume, for example, a system with N cores, each consisting of a 2-D array of $M \times M$ MAC operators; and assume as target workload a sequence of L batched neural network layers, each represented as a matrix multiplication (MatMul) of dimension

$[B \times C] * [C \times K] = [B \times K]$. This workload can be parallelized across different cores under various parallelization schemes: *DataParallelism* replicates the network across the cores and feeds each replication with different data from the batch dimension B ; *PipelineParallelism* assigns different layers L to different cores, such that they can feed each other in a pipelined fashion; and *TensorParallelism* splits a single layer into multiple chunks along the C or K dimensions, and executes each chunk on a separate processor core. Within a core, the tiled MatMul assigned to a specific core is then further unrolled on to the core's internal $M \times M$ MAC array, for example, spatially unrolling the remainder of the K loop along 1-D and the C loop along another one, while mapping the B loop temporally, as illustrated in Fig. 6.

It is, however, important to realize that the selected parallelization strategy strongly impacts the efficiency with which the parallel resources are used. This can be quantified in terms of the *utilization* of the processing elements. As detailed in Fig. 6, the utilization can be broken down into the *core utilization* (ratio of active core count to total core count); the *temporal utilization* (TU) (ratio of compute cycles in which the core is not stalled, versus total clock cycles), and *spatial utilization* (ratio of active MAC units versus total MAC units within one processor core). Without proper mapping, the combination of these three utilization factors can quickly lead to severe compute under-utilization and thus performance far below the theoretical roofline.

Finally, it is important to realize that the increased parallelism in the cores raise the roofline of the core, which shifts the knee point of the roofline to higher AIs [see Fig. 4 (left)], making the system more likely to be memory bound, potentially leading to memory-induced reduction of the TU. Indeed, the execution of a large number of MACs per clock cycle also demands a large data volume to be dragged into the compute array per clock cycle. Although, theoretically, this memory pressure can be resolved by increasing memory bandwidths across all levels of the memory hierarchy to shift the diagonal part of the roofline up, this is often not feasible in practice. ML accelerators often already maximally exploit distributed register files, widely banked SRAM memories, and high-speed (costly) off-chip DRAM, whereby further bandwidth increases come at substantial area and energy costs. Fortunately, it may be possible to increase parallelism without having to proportionally increase memory bandwidth, by employing mappings that leverage increased data reuse, when/if the workloads allow this.

B. Exploiting Spatial and Temporal Data Reuse

The operands of the MAC operations in neural networks have a potential data reuse factor, which can easily go up to 10 000s. For example, the weights in a Conv layer or the operands of a large MatMul operation can be fetched once and subsequently reused in many dot-product operations. With some exceptions (e.g., weights in a fully connected layer with a batch size of 1, or inputs in a depthwise layer), neural network compute kernels, thus, exhibit large intrinsic AI. This, however, does not automatically result in operating at the top-right corner of the roofline model. The processor

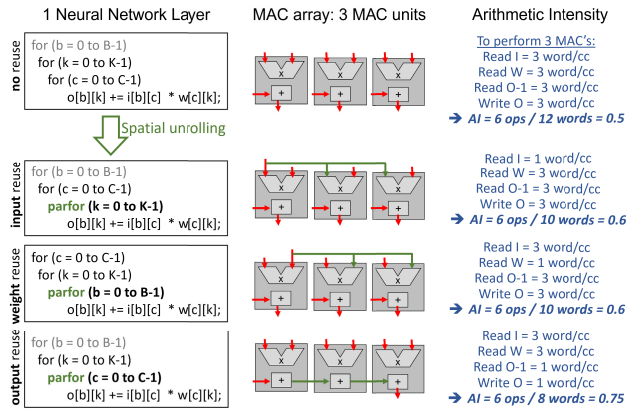


Fig. 7. Spatial data reuse concepts.

hardware has to be able to exploit the kernels' data reuse opportunities at the datapath level. This involves the alignment of the execution schedule to the hardware implementation, in particular, to flexibly leverage both spatial and temporal data reuse as will be explained next.

Spatial data reuse refers to reusing data within a given clock cycle across more than one MAC unit in the datapath. Fig. 7 illustrates how parallel MACs in a MAC array can spatially exploit weight, input, or output data reuse, to increase the effective AI. This requires hardware modifications to enable direct data sharing paths between the hardware units. Without such provisions, the inherent AI of the compute kernel cannot be spatially exploited at the datapath level. All SOTA ML processors exploit such spatial data reuse along one or more operand dimensions of their datapath, typically consisting of hundreds to thousands of MAC units. The examples of Fig. 7 all only contain a single *parfor*-loop (= spatially parallel for-loop), corresponding with 1-D spatial data reuse. This can be found in vector processors, which are typically equipped with a fuse multiply add (FMA) instruction, exploiting output reuse.

However, AI typically does not improve strongly from reuse across a single operator dimension, as the memory accesses of the other operands will quickly start to limit reuse. As a result, ML processors are commonly equipped with 2-D or even 3-D datapaths and thus, multiple spatial *parfor*-loops in their dataflow representation, which jointly exploit different forms of the three spatial reuse concepts along each MAC array dimension. The Envision processor [12], for example, combines 16-way input reuse and 16-way weight reuse, while the Huawei DaVinci cores [20] or Nvidia Tensor cores [21] exploit all three forms in a 3-D MAC array, as illustrated in Fig. 8 (top).

Such multi-dimensional spatial reuse reduces the memory pressure from being linearly dependent on the number of MAC elements A_{op} , to only scaling with $\sqrt[3]{A_{op}}$, substantially increasing AI. More importantly, the larger the compute array is, the more the spatial data reuse can be accommodated. As a result, larger compute arrays can typically operate at higher AIs in the compute-dominated region of the energy roofline [Fig. 3 (right)], which represents the most energy-efficient region of operation. Such very large and efficient compute arrays can be found in recently emerging in-memory compute architectures. Typically, these architectures [Fig. 8 (bottom)]

DaVinci core

```
for (c1 = 0 to ...)
  parfor (b = 0 to 15)
    parfor (k = 0 to 15)
      parfor (c2 = 0 to 15)
        o[b][k] += i[b][c] * w[c][k];
```

Read I = 16*16 word/cc
Read W = 16*16 word/cc
Read O-1 = 16*16 word/cc
Write O = 16*16 word/cc

→ $AI(spatial) = 2*16*16*16 ops / 4*16*16 words = 8$

Analog 256*256 IMC core

```
for (b = 0 to ...)
  parfor (k = 0 to 255)
    parfor (c = 0 to 255)
      o[b][k] += i[b][c] * w[c][k];
```

Read I = 256 word/cc
Read W = [256*256 word/cc]
Read O-1 = 256 word/cc
Write O = 256 word/cc

→ $AI(spatial) = 2*256*256 ops / 259*256 words = \sim 2$

→ $AI(spatial+temporal) = 2*256*256 ops / 3*256 words = \sim 170$

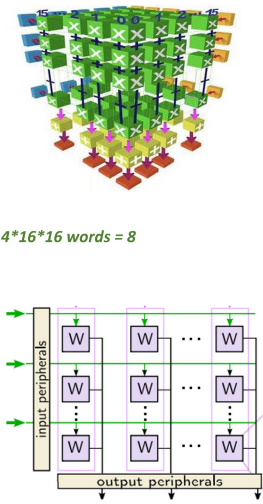


Fig. 8. Spatial data reuse within the DaVinci core [20], and an analog IMC core with 256 rows and columns.

only exploit spatial data reuse across 2-D: while they can massively reuse inputs and output data along the memory rows and columns, each weight is used only once each clock cycle. As long as the weights do not have to be reloaded, this allows them to operate at very high AI. Loading new weights every clock cycle, however, would drastically limit the spatial AI benefits, as numerically illustrated in Fig. 8 (bottom). This is, however, avoided through temporal data reuse, discussed next.

Temporal data reuse can further increase the effective AI. Specifically, it reduces the number of data accesses per compute operation, not by reusing data within one clock cycle (spatial data reuse), but by reusing data *across* clock cycles without repeated memory fetching. When studying the nested for-loops embedding the ML MAC operations (see Figs. 7 and 8), it is clear that for each for-loop, one operand remains constant across different iterations of the for-loop. For example, in the analog IMC core [Fig. 8 (bottom)], the nested for-loop along the “b” dimension reuses the same weight element $w[c][k]$ across all its temporal iterations, as the operand is not dependent on the index $[b]$. This allows us to keep the weights *stationary* across the for-loop iterations, also denoted as a “weight-stationary” dataflow. Such weight-stationary is exploited in almost all in-memory compute cores [22], [23], [24].

Beyond weight-stationary, processors can also be input- or output-stationary, together forming the three orthogonal forms of temporal data reuse. When describing processor dataflow in terms of their nested “for”- and “parfor”-loops, one can easily derive the stationarity of the processor under study. The lowest for-loop above the spatial “parfor” loops determines the temporal data reuse toward the registers. For example, in the DaVinci-example [Fig. 8 (top)], the lowest for-loop is a “c”-loop and thus irrelevant for the output operand. Hence, this represented dataflow is an *output-stationary dataflow*, which is very common in fully digital ML processors. Also, the aforementioned Envision chip [12], Tensor cores [21], and Tesla NPU [14] are output stationary processors. As the

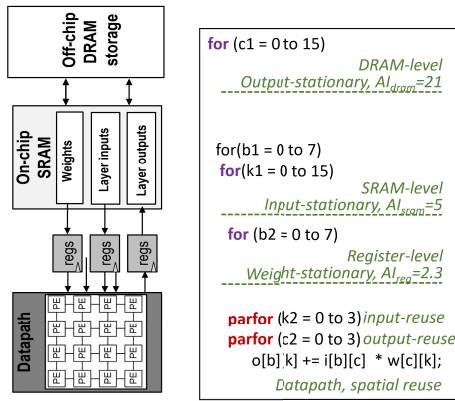


Fig. 9. Nested for-loop representation of workload mapping and tensor allocations across the memory hierarchy.

outputs typically require a larger word length and thus memory bandwidth, compared to weights or inputs, output stationarity (= temporal output reuse) has a significant impact on a processor's AI.

In contrast to spatial data reuse when multiple reuse dimensions can be combined, only a single stationarity dimension can be exploited per memory level. Indeed, it is not possible to keep two operands constant across subsequent MAC computations, as this would lead to repeated, redundant operations. It is, however, possible, to exploit different forms of temporal data reuse across the memory hierarchy. Executing larger tensor operations in a core typically involves tiling the input, weight, and output tensors, corresponding to splitting/blocking different for-loops, and allocating them across different memory levels, as done in Fig. 9.

Here, it is clear that each level of the memory hierarchy can exploit a distinct form of stationarity, as the lowest for-loop tiled within a memory level determines which tensor can remain stationary toward the lower level memory. This also explains why the AI increases when moving from the L1 to L2 and L3 memory levels. Optimal combinations of such spatial and temporal unrolling are, therefore, indispensable toward packing as much as compute for a given memory bandwidth.

C. Exploiting Quantization

Quantization is currently the most actively exploited technique to boost efficiency in ML accelerators. A quantitative estimate of the major performance improvements achieved by GPUs thanks to quantization (16 $\times$) has been reported by Nvidia [25] and is confirmed by extensive literature surveys covering a wide range of accelerators [7], [26], [27].

From a hardware-centric viewpoint, quantization is easily exploitable, and its effect on the roofline, as shown in Figs. 5 and 10, is to raise the achievable peak compute—the height of the compute-bound region—as more operations can be performed faster in the same silicon area, and to increase the AI of an ML workload, as more operations are performed for each byte exchanged with memory. Energy improvements are potentially superlinear because key operators in ML workloads, such as MAC units, have time and space complexity that is super-linear on the bitwidth on their input operands [28].

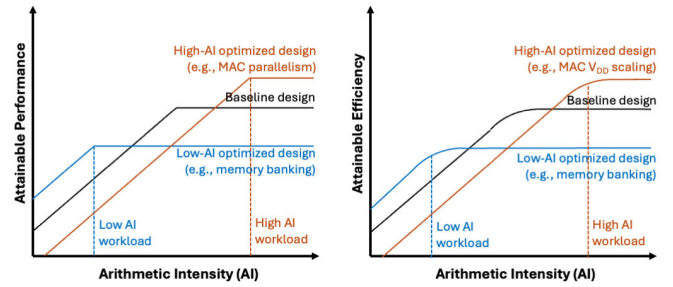


Fig. 10. Roofline optimizations to different workload AIs.

Hence, it is not surprising that the hardware design community has been aggressively pursuing quantization. Recent research results have demonstrated that digital compute efficiency as high as POPS/W can be achieved with extreme, ternary, and binary quantization [29], [30]. On the other hand, such an extreme quantization comes with significant model accuracy loss. The current production models can be compressed down to 4-bit coefficients and 8-bit activations, but algorithmic research is actively working to lower these thresholds [31].

Hardware support for quantization in instruction processors, through dedicated instruction set architecture (ISA) extensions, has been extensively explored, leveraging the well-known single-instruction multiple-data (SIMD) approach. As an example, the RISC-V ISA has been extended to support all mixed-precision combinations of power-of-two subsets of 32-bit registers (namely, 2-, 4-, 8-, and 16-bit operands) [32], [33], as well as binary and ternary operands. Fig. 11 shows the design of an SIMD *expanding*³ dot-product unit for a 32-bit RISC-V core, supporting all power-of-two mixed precision quantized operands below 32 bit. The slicer and router block splits data coming from the 32-b input registers in the appropriate narrow operands and routes them to the MAC units in the dedicated data-path section. Sign-extension is then performed for the smaller operand; multiplication is performed in parallel on 8 $\times$ 5b $\rightarrow$ 10b multipliers, followed by the 8 $\times$ 10b $\rightarrow$ 32b adder tree. Results are stored in the 32-b accumulator register. Note that distinct dot-product units are instantiated for each power-of-two (plus sign extension) bitwidth to allow operand isolation and reduce switching activity.

The SOTA hardware support for quantization is moving along two main axes of evolution: first, fine-grain control of mixed quantization, where large multioperand operators, such as large multiplier arrays followed by adder trees for dot-product computation, are designed to enable different and configurable bitwidth for different classes of operands (activations versus weights) [25]. Second, *block-quantization* support is getting increasingly emphasized, as operand-by-operand (scalar) quantization has plateaued [34]. Block-quantized data formats for ML are also being standardized in an effort to facilitate model portability across different hardware platforms: the microscaling standard formats [34], proposed by NVIDIA, AMD, Intel, Microsoft, Meta, Arm, and Qualcomm in the open

³This denomination stems from the fact that the accumulator is computed and stored at higher precision than the operands, 32 bit in this case.

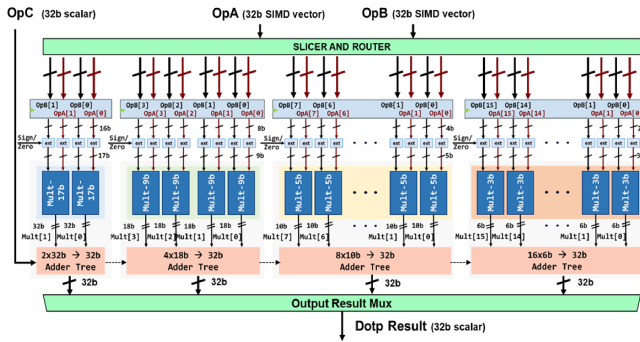


Fig. 11. RISC-V mixed-precision SIMD dot-product unit.

compute project (OCP), are specified down to 4-bit floating point for blocks 32 or 64 numbers, with a shared 8-bit exponent. Fig. 12 provides a visual summary of block quantization as standardized by OCP and depicts the trajectory from simple scalar and uniform quantization to advanced mixed and block quantization schemes. The most recent GPU generation from NVIDIA, Blackwell, supports all these formats, including the aggressively scaled MXFP6 and MXFP4 [35].

A representative design of a specialized tensor accelerator with finely tunable block quantization support for weights is Neureka [36], as shown in Fig. 13. Activations and weights are read in blocks through wide ports from L1 memories (not shown in the figure). Blocks are buffered in internal (L0) latch-based register files, which are preferred over SRAMs because the required capacity is low and energy-per-access can be minimized more aggressively than with SRAMs by voltage scaling, which can be pushed further for logic cells than for SRAM macros. The datapath is designed to access bit slices of each weight block. Multiplication of bits of the weights, with 8-bit activations, is performed, one bit at a time, and products are accumulated into multi-bit numbers (12 bits), which are then locally shifted over a 20-bit range. The shifted output of each COL combinational datapath is then further accumulated by a larger adder tree that collects multiple local shifted sums. Results are then stored in output accumulation registers in an output-stationary dataflow.

This datapath can handle weights quantized between 2 and 8 bits. The number of cycles needed to compute the final output-stationary summation grows linearly with N_w , where N_w is the number of bits used to represent weights. Weight precision is temporally unrolled, and activation precision is spatially unrolled. The bit-by-bit shift and accumulate steps can be seen as one additional dimension in the nested for-loop notation discussed in Section III: this is shown in Fig. 14. In this example, Neureka's datapath accumulates in parallel over inputs and output channels and convolutional window size, but sequentially over the bit of the weights.

Note that Neureka does not offer tunable activation quantization: this extra degree of freedom has also been explored in the literature [33], but quantization of activation below 8 bits is usually not accuracy-neutral. Closing this accuracy gap at a high level of robustness is an active research field.

An implementation of Neureka in 16-nm TSMC technology, featuring 36×32 COL units achieves end-to-end (including

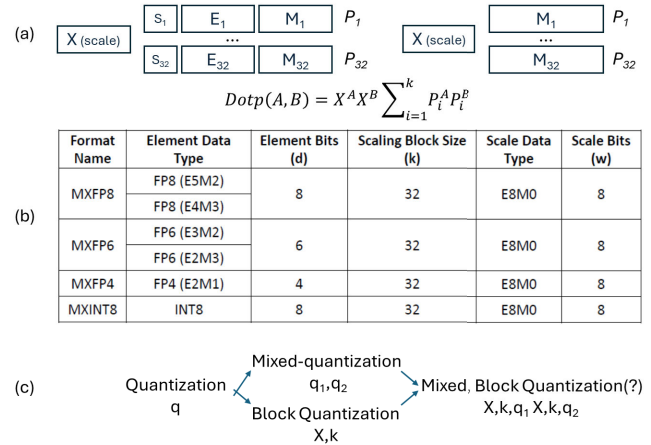


Fig. 12. (a) Block quantization and dot product of block-quantized vector fragments, (b) OCP microscaling specification of block-quantized numbers, and (c) from basic quantization to mixed and block quantization, and future block and mixed quantization.

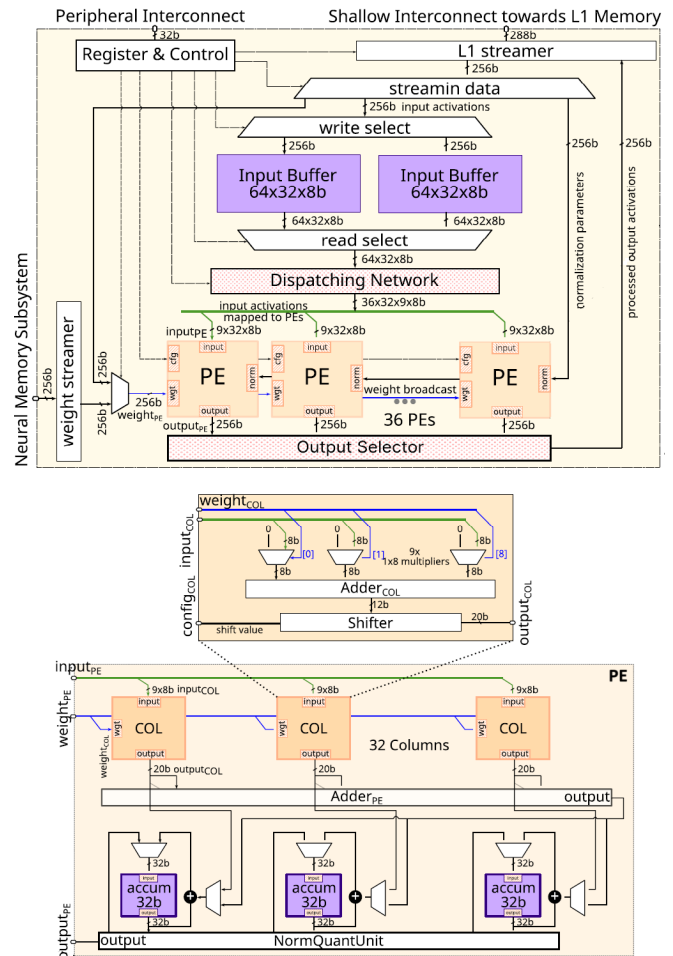


Fig. 13. Neureka mixed-precision accelerator architecture design.

L1 memory access cost) energy efficiency ranging from 8.84 (2-bit weights) to 2.68 TOPS/W (8-bit weights) at 0.6 V [36]. Note that energy scaling is not perfectly linear in the number of bits because a fraction of the execution cycles does not scale with weight bitwidth: a few cycles are needed for scale and offset-related computation. Nevertheless, a significant energy efficiency boost is achieved ($3.5\times$ when scaling weight bitwidth by a factor of 4), by aggressively quantizing weights.

```

for(k_out=0 to 32):
  for(w_bit=0 to 8):
    par_for(h_out=0 to 6):
      par_for(w_out=0 to 6):
        par_for(k_in=0 to 28):
          par_for(fs=0 to 9):
            load_weight(wgt[k_out, k_in_tile, w_bit, fs, k_in])
            accumulate[h_out, w_out, k_out] += inp * wgt
          load_features(next_tile)
        normalization()
        quantization()
      streamout()
    
```

3x3 Conv

Fig. 14. Nested loop for execution of a 3×3 convolutional layer in Neureka. Note that shift-and-add loop is executed sequentially bit-by-bit for the 8-bit weights.

D. Exploiting Sparsity

Sparsity is present in most classical and advanced ML models, as coefficients tend to cluster around zero after training. As a consequence, activations are also pulled close to zero, albeit with a smoother distribution. Sparsity exploitation is also one of the key efficiency boosters for incumbent GPU architectures [25], although to a lesser degree compared to quantization ($2\times$ versus $16\times$). On the other hand, the impact of sparsity is likely to grow as the trend is toward large-scale models that are intrinsically highly sparse, not just as a result of training. *Mixture-of-experts* models [7] are notable examples of this trend.

Hardware support for extremely sparse models with non-structured sparsity is being explored, as the rapidly growing model size trend may soon impose the use of indexed data structures to handle high degrees of sparsity for the largest “leadership” models. An example of the accelerator with hardware support for highly sparse tensor computation is the Onyx coarse-grained configurable array (CGRA) architecture [37]. The sparse data structure supported by Onyx is known as *fibertree*, and it is depicted at the top of Fig. 15. The fibertree is a 2-D-indexed data structure that indexes non-empty rows (or columns) and for each one of them, it indexes the respective non-zero elements. Only indexes and non-zero data elements are stored in memory, thereby achieving large compression factors if sparsity is high and non-structured.

For efficient execution in the Onyx accelerator, the fibertree is encoded, as shown in the bottom of Fig. 15, and in streams of indexes, it read sequentially by the hardware engine, which ingests parallel streams for the rows/columns of the tensors involved in the operation being targeted. In the case of the intersection datapath, used for instance for computing the dot products of rows with columns needed for MatMul, only equal indexes in both streams (coord_1 and coord_2) are output by the intersector (coord stream), and the corresponding data elements are then loaded from memory for product calculations, which are then locally accumulated.

While the reduction of memory bandwidth to access data is apparent (only the strictly needed data elements and their indices are loaded), access to the non-zero elements has lower block locality compared to dense tensors; hence, efficiently loading data from remote, off-chip DRAM memories, which generally benefit from burst access to blocks of continuous data, is a challenge. Thus, most hardware optimization efforts

have focused on *structured sparsity*. $N:M$ sparsity is widely supported, where a maximum of N nonzero elements is enforced over an array of M elements [38]. $N:M$ sparsity is hardware friendly, especially if N and M are design-time constants: indexes and data have constant size, and efficient block transfers from/to memory remain viable. As a result, many production ML acceleration engines, most notably GPUs, support $N:M$ sparsity [39] (only 2:4 sparsity is supported natively in hardware). On-going efforts are exploring more advanced structured sparsity models, for instance, multidimensional sparsity for tensors [40].

Analyzing sparse workloads and architectural optimizations on the roofline is non-trivial. Fig. 16, derived from [10], exemplifies the challenge. All the blue symbols correspond to a parallel tensor contraction kernel executed on the A100 GPU at different levels of sparsity. The dense version of the kernel is very close to the roofline in the compute-bound region, indicating high utilization and architectural efficiency. The version exploiting structured sparsity is still close to the roofline, but slightly more detached and moved to the left (toward the memory-bound region). This is motivated by the significantly reduced number of operations effectively executed (only those with non-zero operands in both tensors) and the less aggressive reduction in memory accesses. The latter is caused by the need to load the index of non-zero elements, as well as the fact that non-zero elements in one tensor can be loaded and then not used if they do not correspond to non-zero elements in the other tensor. Furthermore, the reduced burstiness of memory accesses due to the smaller size of the tensor tiles in compressed form makes it harder to fully exploit the wide transfers at the main memory interface (HBM). Finally, the highly sparse (non-structured) version of the kernel uses indirection, which increases the number of short-burst memory accesses and decreases the AI further. The result is a reduction in the number of effective GOPS that can be executed per second (because of the extra memory stalls and bookkeeping instructions needed). This implies a shift toward the memory-bound region of the roofline and a very significant detachment from it, due to under-utilization of the peak memory bandwidth.

Architectures with advanced support for sparsity, both hardwired (see [37]) and flexible (e.g., Symphony [10] and Occamy [41]), corresponding to the red dot in Fig. 16, can significantly reduce the gap from the roofline, thanks to hardware-accelerated indirection. Unfortunately, the kernel remains in the memory-bound region of the roofline because hardware optimization does not change the number of executed products and sum operations with respect to the memory accesses. The effective AI (computed with only the non-zero memory accesses and non-zero operations) remains lower than for the dense workload, signaling less opportunities for data reuse. Hence, if we look at the roofline alone, it may seem that sparsity exploitation is not a good idea, as it reduces the effective GOPS/s. However, the very significant reduction of operations may still lead to absolute energy and latency advantages with respect to ignoring sparsity, using only dense kernels. Roofline analysis alone, albeit useful, is not sufficient to ascertain the overall end-to-end task speedup and energy

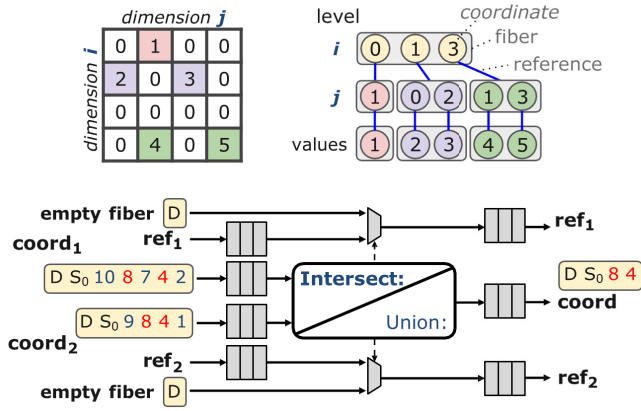


Fig. 15. Onyx's fibertree sparse tensor format and hardware unit for intersection and union operators [37].

efficiency improvements, especially when contemplating the addition of dedicated hardware acceleration for unstructured sparsity. Recent literature results [10] indicate that efficiency improvements exceeding one order of magnitude are possible through dedicated hardware if tensors are very sparse (non-zero density below 0.39%). Current ML models are quite more dense than that. Hence, while structured sparsity is immediately exploitable for current models with some tuning effort, non-structured sparsity requires model evolutions toward higher level of sparsity.

It is also important to note that quantization and sparsity may be seen as two facets of the same data representation problem: we conjecture that in the future they will be combined in advanced data representation schemes with hardware support aiming at more aggressively reducing the number of bits needed, on average, to store and manipulate weights and activation. On the other hand, the more aggressively quantized are the elements of a tensor, the harder is to get additional advantages by sparsity exploitation (since indexing overhead becomes relatively larger). Optimally balancing sparsity and quantization with efficient hardware, locally and at the level of global memory access, is an interesting and open research challenge.

E. Exploiting Near-/In-Memory Compute

The need to simultaneously address both the efficiency over a large number of operations and the data-movement costs across large amount of data has motivated near-memory computing (NMC) and in-memory computing architectures. Fig. 17 shows the progression to such architectures from traditional computing architectures.

As already considered in detail, the traditional architecture is based on the separation of a large amount of computation from a large amount of memory, with the hardware for each sized to provide the horizontal (compute-bound) and diagonal (memory-bound) rooflines. As we boost these rooflines, the physical size of both hardware structures also scales, leading to substantial data communication costs (energy and latency) during computations. Representative energies for communicating data out of the memory and from the memory to the processor are shown in Fig. 17 for reference.

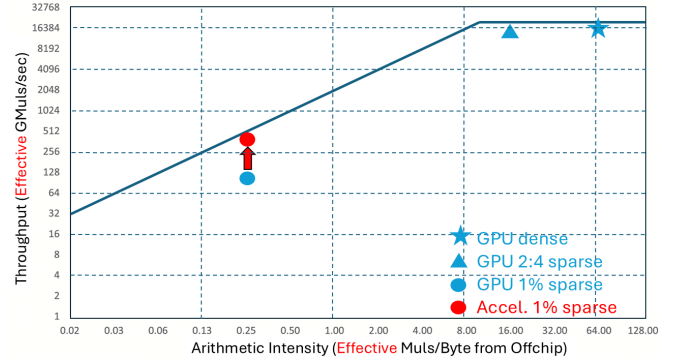


Fig. 16. Structured and unstructured sparsities—roofline analysis based on data from [10]. The red dot corresponds to the symphony architecture.

The NMC architecture is based on finer-grained partitioning and proximate placement of the memory and computation hardware, so that the data-communication energy efficiency and aggregate bandwidth are substantially enhanced. This yields an upward shift in the diagonal roofline. Again representative energy numbers are shown for reference. Such an architecture is amenable under two conditions: 1) the computations involve substantial parallelism, so that the computing hardware can be partitioned accordingly and 2) the data required for different parallel computations is well-structured and exhibits high locality, such that it can be readily stored in the partitioned memory associated with each parallel computation. As mentioned, both of these conditions are fortunately typical in ML workloads (as well as a range of other signal-processing workloads), making various forms of NMC a natural and commonly employed evolution on traditional integrated architectures. However, the NMC architecture has also been adopted within memory-centric (e.g., DRAM) technologies, providing integrated computation for data reduction over energy- and bandwidth-limited communication channels, opening new processor-memory architectural options and tradeoffs (not discussed in detail here) [42], [43].

The in-memory computing (IMC) architecture eliminates the explicit separation of memory and computation hardware, and in doing so, eliminates the costs of both communicating data out of the memory and of communicating data from the memory to a processor. In doing so, IMC introduces the potential for substantial efficiency and performance advantages, but through a number of critical circuit and architectural tradeoffs that impact computation accuracy and hardware utilization. While NMC is a straightforward evolution on traditional architectures, IMC presents a number of differentiating considerations and is the focus of Section III-F.

Before analyzing its operation and tradeoffs, we note that IMC can be seen as an extreme case of 2-D spatial architectures. Such architectures leverage a 2-D arrangement of PEs, each providing local data storage and computation, in order to exploit 2-D data reuse in matrix operations. In IMC, PEs are reduced to high-density memory bit cells, and the opportunities presented for reducing data communication can be viewed by considering each of the operands involved in a matrix operation. For inputs and outputs (activations) provided to/from the IMC macro, high-density memory bit cells enable

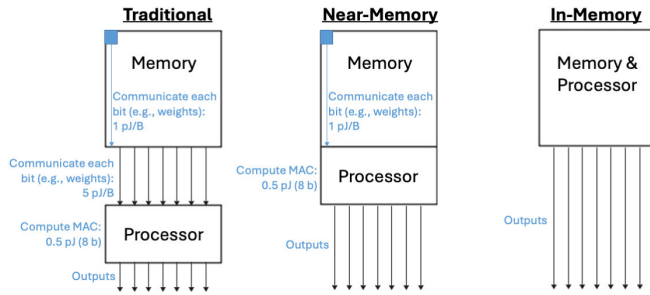


Fig. 17. Comparison of traditional, near-, and in-memory architectures.

greater spatial data reuse, by virtue of a greater number of PEs possible within an allocated area. For weights stored in the IMC macro, the integration of computation capabilities within bit cells eliminates explicit accessing, at least from a first-level memory, thereby eliminating the roofline diagonal associated with the first-level memory. As described in the following, IMC architectures typically require the use of additional level-2 memory for AI model scalability. Thus, viewing it as a high-density PE array (enabling greater parallelism within an allocated area) with enhanced energy efficiency (enabled by analog or custom-digital hardware), IMC has two effects on the roofline model: 1) it boosts the compute-bound roofline and 2) enables increased levels of reuse, moving workload operating points further into the compute-bound regime.

F. In-Memory Compute Tradeoffs and Approaches

A range of circuit approaches have been proposed for enhancing the energy efficiency of computation, and a number of these have been explored within IMC architectures, as mentioned further below. On the other hand, IMC fundamentally addresses data movement through a specific approach, namely, by performing in-memory data reduction. In doing so, IMC fundamentally instates a dynamic-range tradeoff versus energy efficiency and compute density.

While IMC has been proposed for a range of compute operations, matrix-vector multiplications (MVMs), which are of particular importance in ML workloads, provide the most direct opportunity for such data reduction. This arises because MVMs first involve parallel operations, namely, multiplication between input-vector elements and matrix weights, followed by a reduction operation, namely, accumulation along the matrix inner dimension. Fig. 18 shows a typical in-memory computing architecture. Input-vector elements are provided in parallel to the memory rows, via the word lines (WLs) or other dedicated inputs, realizing spatial input reuse. The bit cells, either via existing or added circuitry, then perform multiplication with stored data. Finally, accumulation is then performed along each column, on the bit lines or other dedicated outputs, realizing spatial output reuse.

IMC designs have covered a broad range of approaches to this basic operation. Input data have been provided through different signal-modulation schemes, leveraging pulse-amplitude modulation [44] and pulsewidth modulation [45]. Bit-cell storage has been provided through binary [44], [45] and multi-level [46], [47] cells, while multiplication and accumulation has been performed through

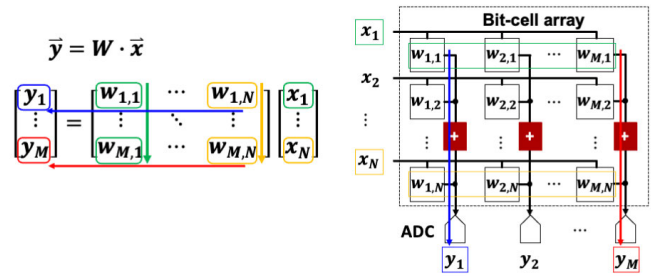


Fig. 18. Basics of in-memory computing for MVM operations.

digital [48], [49] and analog [24], [44], [46] circuits. Finally, output readout has been accomplished by direct digital representation and analog-to-digital conversion from the current [50], voltage [51], and time [52] domains. In all cases, IMC instates specific fundamental tradeoffs, constrained by the storage and compute technologies employed, and mediated by critical design parameters described next.

1) *IMC Fundamental Tradeoffs*: From its basic operation, we see that IMC derives its gains from high-density PEs (bit cells), providing in-memory data reduction. Fig. 19 analyzes the critical IMC tradeoffs determining the achievable gains, compared with traditional/NMC architectures, mediated by the row parallelism P_R (where P_C is the column parallelism). Simplifying the more detailed analysis in [53], we see that a traditional architecture requires energy $E \times P_R$ and latency $L \times P_R$ (from P_R switching cycles), while IMC requires only E and L energy and latency, respectively. However, while a traditional architecture accesses individual bits of stored data having dynamic range D , IMC accesses a computed result via accumulation over P_R bits of data, causing dynamic range increase to at least $D \times P_R$.

Generally, maximizing density and supporting increased dynamic range has energy and performance, as well as, notably, signal-to-noise ratio (SNR) implications, particularly when leveraging analog operation. This has direct impacts on ML model accuracy, as we also previously saw with quantization, and establishes a critical tradeoff space for IMC. Quantitatively, the tradeoffs depend on the underlying technologies employed for data storage and computation, with implementations, thus ranging from low levels of row of parallelism (two–three rows, where IMC gains are minimal) [54], moderate levels of row parallelism (tens of rows) [55], and high levels of row parallelism (1000s of rows) [24]. We, therefore, see that the choice of underlying technologies is critical to the gains possible from IMC.

It is important to note that the discussion above frames the IMC dynamic range tradeoff as mediated by the row parallelism P_R . However, the dynamic range following in-memory data reduction (accumulation) also depends on the dynamic range of multiplication operands D_X and D_W , inputs, and weights, respectively,

$$D_y = (2^{D_x} + 2^{D_w} - 1) \times P_R. \quad (6)$$

Since multiplication precedes the critical IMC accumulation, its contribution to the dynamic range can be managed outside of the IMC macro, through bit-sliced operation, where multiple input bits are processed in a bit serial manner and

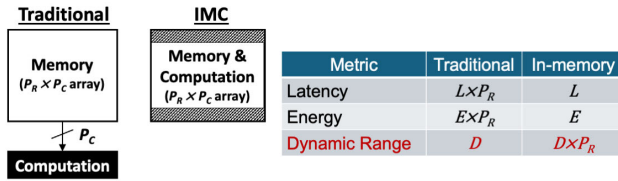


Fig. 19. Fundamental dynamic-range tradeoff of in-memory computing [53].

multiple weight bits are processed in a bit-parallel manner. This causes energy and throughput scaling as typically expected in multipliers (i.e., linearly with each operand). Though it changes how IMC accumulation impacts SNR, SNR and thus model accuracy ultimately remain limited by the IMC accumulation operation [51], [56].

2) *Technologies and Implementations*: Fig. 20 considers a generalized implementation model for an IMC column, comprising: input drivers, for providing row-parallel input data, X ; bit cells, for storing weight data W and performing multiplication with inputs; mechanism for accumulation; and readout circuitry, for providing digitized outputs. The maximum total dynamic range of the system (in bits) is set by the sum of input bits, B_X , and weight bits, B_W , involved in each multiplication, plus $\log_2$ of the number of rows involved in accumulation P_R . The overall dynamic range may be limited by any of IMC circuit components, as discussed in the following.

There have been two broad approaches to IMC design: digital IMC (D-IMC), where the critical accumulation operation is performed using an adder tree, based on digital logic, applied across all memory rows [48], [49], and analog IMC (A-IMC), where the critical accumulation is performed using analog current or charge, followed by analog-to-digital conversion [53]. Due to the efficiency with which digital circuits can support dynamic-range scaling, D-IMC can readily preserve computation SNR and thus model accuracy. However, by employing the same fundamental technology as digital accelerators, it provides only incremental gains in energy efficiency and compute density relative to traditional digital architectures. As shown in Fig. 21, D-IMC architectures are typically dominated by their digital circuitry, such that the MVM-level gains achieved relative to traditional digital architectures are in the range $2\times$, arising primarily due to the use of custom-layout adder and weight-storage logic circuits. To reduce the digital circuitry and increase data-storage density, recent D-IMC designs have attempted to time-share the computation logic across multiple sets of bit cells, but where the original compute density and energy efficiency remain largely unchanged due to temporal operation [57].

On the other hand, A-IMC has shown the potential for substantially higher energy efficiency than digital architectures [58], but has primarily been limited in SNR by various sources of analog noise. Many A-IMC approaches have attempted to scale the operation of typical memory cells, which communicate a stored bit via an output current, in order to minimize the need for additional devices. However, A-IMC implementations based on SRAM [44], flash [46], resistive random access memory (ReRAM) [55], magnetic random access memory (MRAM) [52], [59], and phase-change memory (PCM) [60] have all shown that the bit-cell

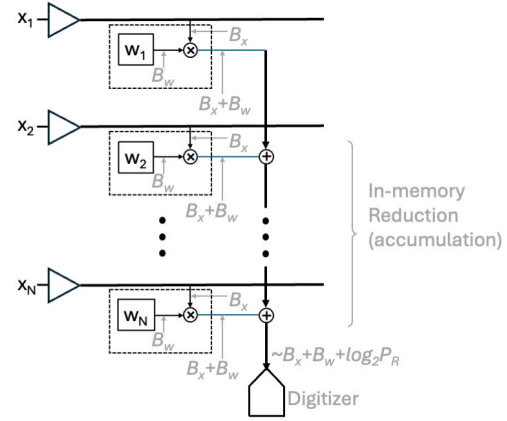


Fig. 20. Generalized column computation in in-memory computing.

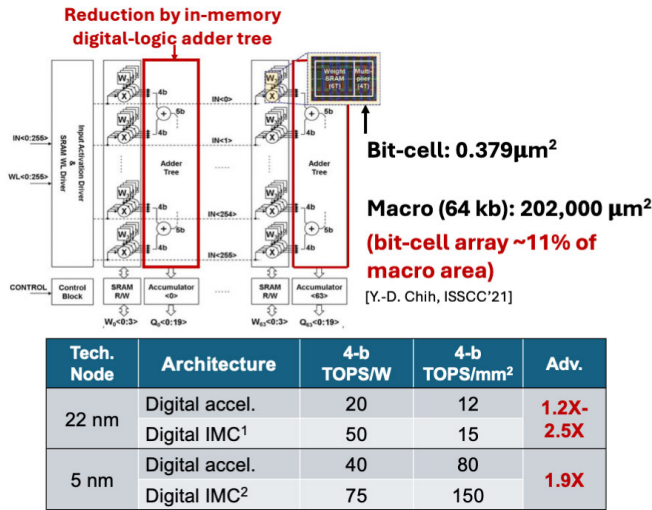
current variation and nonlinearity introduce substantial SNR and model-accuracy degradation, preventing scaling from single-bit outputs to the larger dynamic range following accumulation, required for IMC efficiency and throughput advantage.

Emerging resistive embedded non-volatile memory (eNVM), particularly ReRAM [61] and MRAM [62], which have recently begun to be available in foundry CMOS technologies, have been of particular research interest due to their potential for density scaling in advanced CMOS nodes. However, these technologies tend to provide reduced IMC signal, due to low ON-to-OFF-resistance ratios, in the range of $2\times$ to $10\times$ (i.e., well below those of MOSFETs used in SRAM technologies, in the range of $10^4\times$), and low absolute resistance. With high IMC row parallelism, this leads to low SNR and high current for voltage sensing, thus leading to poor readout-circuit power and area efficiency, often dominating over the memory array itself. This has prohibitively limited the practical efficiency and compute density achievable by eNVM-based IMC.

The fundamental SNR tradeoff in IMC has instead motivated switched-capacitor (SC) operation [24], which moves away from current-based signaling of individual bit cells, as prominently used in standard memory operation. As shown in Fig. 22, SC IMC leverages highly precise and temperature-stable lithographically processed backend-of-line (BEOL) metal capacitors, formed above the bit cells, to enable high SNR charge-domain accumulation. Analysis has shown that this can enable row-parallel operation in the tens-of-thousands, with a practical demonstration of over 4k-row implementations [24]. Further, voltage output signals from SC operation enable direct feeding of energy- and area-efficient ADC architectures (e.g., capacitive SAR), but still impose energy and area overheads ranging from 15% to 40% [51], [63].

SC IMC has led to complete and scalable full-architecture demonstrations, integrating A-IMC with digital infrastructure (accelerators, CPUs, on-chip networks, and level-2 memory) [64], as well as the highest efficiency IMC macros to date (up to 120 TOPS/W for 8-b computation in 28-nm CMOS) [63].

3) *Architectural Implications*: At the architectural level, a critical challenge introduced by IMC is that it intrinsi-



¹[Y.-D. Chih, ISSCC'21], ²[H. Fujiwara, ISSCC'22]

Fig. 21. Analysis of digital in-memory computing versus standard digital architectures.

TABLE I

ANALYSIS OF IMC, OPTIMIZING FOR STORAGE VERSUS COMPUTE UTILIZATION IN A FULLY WEIGHT-STATIC MAPPING, CONSIDERING UTILIZATION LOSSES JUST DUE TO THE CONFLICT BETWEEN THESE

Model	Params	Mapping w/o Replication		Mapping w/ Replication		
		IMC bits	Storage Utilization	Compute Utilization	IMC Bits	Storage Utilization
ResNet-50	23.9 M	190 M	1.00	0.19	610 M	0.31
ResNet-101	42.8 M	342 M	1.00	0.18	1,216 M	0.28
BERT	110 M	1,425 M*	1.00	1.00	1,425 M*	1.00
UNet-2D	31 M	248 M	1.00	0.017	1,704 M	0.14

*Includes computed weights

cally couples storage and compute resources, yet maximizing utilization and scalability require optimizing each of these separately. This prevents a fully weight-static workload mapping. To illustrate Table I, consider fully weight-static mapping optimizing storage utilization and compute utilization, respectively, while analyzing the isolated effect of utilization losses that result from the conflict between these (ignoring all other sources of utilization loss, i.e., assuming perfect input activation feeding). Columns 3–5 consider workload mappings that optimize for storage, by storing each of the model weight bits statically in only one IMC bit cell. Since weights are generally involved in a highly differing number of operations depending on the model architecture, this can lead to extremely low compute utilization, due to TU loss in IMC bit cells storing weights involved in fewer operations. As shown, while transformers (e.g., BERT), comprised primarily of dense layers, maintain a uniform number of operations for all weights in the model and thus preserve high compute utilization, convolutional backbones, including ones used in generative vision models (e.g., UNet-2D), exhibit widely varying operations, thus substantially limiting compute utilization across workloads. Conversely, columns 6–8 consider workload mappings that optimize for compute utilization, by replicating weight bits in IMC bit cells in proportion to the number of operations. This requires the excessive number of IMC bit cells shown, effectively leading to low spatial storage utilization across workloads, thus substantially limiting architectural scalability.

Such analysis illustrates the need to ultimately break the compute-storage coupling through the use of level-2 (L2) memory, so that workload computations can be temporally mapped, to address architectural scalability while ensuring high utilization. This has important implications at the level of IMC circuit and technological design, as well as at the level of architectural design. In terms of circuit and technological design, for instance, IMC write energy and bandwidth become critical for efficient temporal mapping of workloads, limiting the applicability of eNVM technologies, due to their write energies and endurance. In terms of architectural design, the need for efficient spatial (parallel) and temporal mapping of workloads invokes the many considerations discussed earlier in this article, but now optimized to the distinct circuit and microarchitectural attributes of IMC, in terms of area, compute throughput/energy, and weight-loading bandwidth/energy. Architectural research in IMC has begun to consider such factors. Doing so has exposed the overheads of temporal execution and different forms of parallel execution (data, model, and pipeline), which reduce the achievable compute rooflines, and also motivated specialized architectures to minimize such overheads [64].

IV. ROOFLINE MODELS FOR TRADE-SPACE EXPLORATION

The roofline model represents salient aspects of an architecture, illustrated by the various ways the architectural techniques discussed in Section III either raise the performance or energy rooflines, or enable workloads to more closely approach the performance or energy rooflines. This implies that, in addition to characterizing a given architecture, the roofline model can serve as a useful tool for architectural design-space exploration and analysis. This section examines how the roofline model can drive architectural decisions, by considering how different architectural tradeoffs impact the roofline model. Ultimately, the workload-level performance and efficiency across a relevant set of workloads would be determined by combining the roofline model with the workload mapping, which impacts the AI and utilization.

As it is not straightforward for a designer to find the best tradeoff between these conflicting design considerations, several performance modeling and design-space exploration frameworks have been appearing in the SOTA [65], [66], [67], [68], [69], [70], [71], [72]. These frameworks allow to model many hardware architecture variants and can estimate the execution cost of running specific workloads on them, while optimizing the efficiency of the spatial and temporal mapping through loop optimizations such as unrolling, tiling, and reordering. Nonetheless, roofline models provide a valuable tool for building intuition and rationale underpinning the results of such analysis frameworks.

A. Compute Versus Memory Area Allocation

The methods previously suggested for raising the compute-bound and memory-bound rooflines directly impact area and energy, i.e., MAC parallelism and voltage scaling in the case of compute-bound performance and efficiency, and memory banking in the case of memory-bound performance

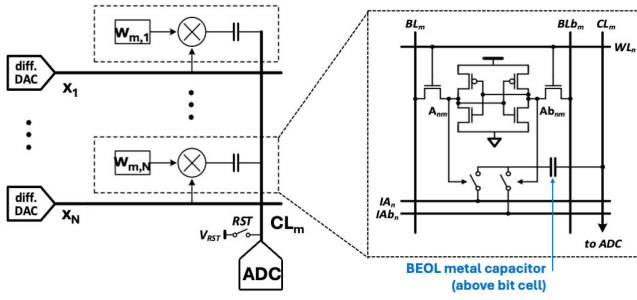


Fig. 22. SC in-memory computing for high-SNR computation.

and efficiency. This suggests an approach to architectural optimization under a joint area and energy constraint. As shown in Fig. 10, the level of compute parallelism and memory parallelism (banking) can be optimally set to the AI presented by a target workload. The critical challenge, in practice, is that architectures must typically be designed to support a range of workloads with differing AIs, and where programmability overheads and associated utilization losses will further reduce actual efficiency and performance.

Of recent relevance, the explosion in ML model size has made the total memory capacity required a critical challenge even at the level of off-chip DRAM, thus extending critical memory-bandwidth and compute-parallelism tradeoffs beyond a single chip. For instance, the capacity limitations of contemporary DRAM high-bandwidth memory (HBM) necessitate executing LLMs across multiple chips, each with attached HBM. This can require interconnection between multiple (16–32) chips, making inter-chip networking an additional key technology, together with compute and memory technologies. In fact, to simultaneously meet memory bandwidth and capacity requirements, architectures have been proposed based only on high-bandwidth on-chip SRAM, integrating several hundreds of megabytes on chip and requiring over 500 chips to be networked even for LLMs of modest size [73].

B. Parallelism Versus Utilization

While the previously discussed approach of employing large processing arrays raises the roofline by maximizing parallelism and minimizing the energy per operation, the challenges inherent with spatial, temporal, and core utilization presented in Section III-A can often prevent operation near the roofline. To analyze workload-level performance, the roofline model must be considered together with the workload operating point, which itself is determined by the workload mapping. As we will see, architectural decisions favorably impacting the roofline can adversely impact the workload operating point.

For instance, larger spatial arrays make it much harder to maintain full spatial utilization across a wide range of workloads. For example, if we map a workload with only 128 output channels on the analog IMC core of Fig. 8 (bottom), only 32 768 of the 65 536 MACs will be activated during a typical compute cycle, resulting in a spatial utilization of just 50%.

Additionally, large monolithic processing arrays require a massive amount of data per clock cycle, limited by memory bandwidth. The severity of such an effect depends on the workload mapping, where leveraging greater levels of data reuse

enhances AI, pushing back to the compute-bound regime. Larger arrays may enable increased data reuse by virtue of storing more data involved in the operations, but the maximum level of data reuse is ultimately limited by the workload itself. For example, if we have to reload the weights of the IMC array of Fig. 8 (bottom) every 1024 cycles, and this weight loading takes 256 cycles (one row per cycle) during which no compute can take place, one-fifth of the clock cycles is lost to weight transfers, resulting in peak TU of just 80%. The total utilization per processor core (the product of the spatial and the TU) determines the distance between the roofline peak performance and the obtained performance within that core. Hence, utilization puts a limit on how close the processor can approach the roofline performance [Fig. 4 (right)].

These tradeoffs become particularly critical with IMC architectures, due to their parallelism tradeoffs and relative efficiency of compute versus weight transfers. In terms of parallelism, as previously discussed, IMC derives its efficiency advantages from high levels of row parallelism and compute-density advantages from the integration of many dense bit cells within each core, making spatial (bit cell) utilization loss a key challenge. In terms of weight transfers, while row-parallel data reduction improves compute efficiency and throughput in IMC, weight loading remains limited to bit-by-bit writing, making TU losses of increased severity. Initial demonstrations of IMC architectures have begun to address these challenges, by noting that different forms of parallel execution (data/model/pipeline-parallelism) yield different tradeoff points, thus integrating low-overhead supports to optimize across different forms of parallel execution [64].

The fundamental tradeoff stems from the fact that larger monolithic PE arrays (e.g., large IMC arrays or widely parallel digital tensor cores operating at reduced precision) are good from a roofline point of view, but challenging from a utilization point of view. Smaller tensor cores, on the other hand, are easy to keep well-utilized both spatially and temporally. Yet, they can only exploit limited data reuse with low AI, resulting in a lower peak array performance. Such a system would have a lower roofline, which can be approached quite closely across more workloads.

Similarly, architectural supports for sparsity inevitably cause overheads in the computational and memory-accessing control flow, yielding reduced roofline. However, when sparse operations are analyzed as no-operations, such architectures yield substantially higher utilization compared to architectures with no sparsity support, where the no-operations directly result in utilization loss. In this way, roofline models expose the benefits of low overhead approaches to sparsity support within an architecture (as discussed in Section III-D and Fig. 16), by minimally reducing the roofline while enabling high workload utilization (operating points near the roofline).

C. Flexibility (Programmability) Versus Specialization

All the previous sections have described approaches that involve some degree of tailoring hardware to the ML application domain. Hence, the game of increasing efficiency is mostly a game of specialization. The multi-order-of-magnitude efficiency boost associated with domain specialization has been known for a long time, well before the ML hardware

explosion [74], [75]. However, there are two unique differentiating factors associated with AI that favor specialization. First, the application markets are huge, leading to large potential production volumes and margins, which can justify the non-recurring engineering cost of a specialized architecture. Second, most ML models are dominated, from a computational standpoint, by relatively few kernels, namely, linear and few non-linear tensor operators [76]. Furthermore, for all deep neural models, the arithmetic precision needed for these operators can be much reduced, as shown in Section III-C. The combination of these factors has led to a “specialization gold rush” with a proliferation of ultra-specialized accelerators.

It is important to note, however, that even in this unique context, hyper-specialization is a dangerous slope, with the pitfall that aggressively specialized NPU architectures may achieve the lowest energy-per-operation at peak utilization, but are subject to under-utilization when workloads change and evolve [76]. Hence, a poorly utilized, ultra-specialized NPU can become less efficient at the workload level than a highly utilized, but less specialized, architecture (e.g., a GP-GPU). If we visualize workloads as points on the roofline plot, we ideally would like our architecture to have as many points as close as possible to the roofline.

A concrete example of such workload evolution is the recent transition of leading-edge models from convolutional neural networks to attention-based networks (transformers). From the computational standpoint, the key operator in CNNs, namely, the N -channel 2-D convolution, is a high-locality kernel exemplified in Fig. 14. Convolution can be transformed in general MatMul through a Toeplitz transformation, with a non-negligible memory inflation [77]. Hence, several acceleration engines have been designed for executing convolutions natively, with no storage overhead. However, the new widespread transformers are dominated by MatMul in the attention block, and thus, hardware that is specifically designed for convolutions faces critical limitations.

Hence, achieving a high utilization across workloads, including unknown future ML workloads, will require flexible architectures, capable of adjusting at compile-time or run-time to the workload characteristics. Although it is difficult to quantify flexibility, without knowing the complete kernel space of interest, recent work defined four possible axes of flexibility, denoted as TOPS [78]: tile size (= sizes of the temporal loops); order (= relative ordering of the temporal loops); parallelization (= which dimensions to unroll in the spatial loops); and the shape (= sizes of the spatial loops). These metrics allow us to compare the flexibility of existing and envisioned accelerators in terms of their GeMM support.

Workloads can, however, evolve in other directions than purely in terms of their GeMM operations with new alternative operators popping up. This leads us to another significant risk factor in specialization linked to the well-known “Amdahl effect”: if a fraction of a workload $f < 1$ cannot be accelerated, the end-to-end speedup that can be achieved by the best ideal accelerated architecture is upper-bounded by $1/f$. Hence, to achieve high end-to-end speedup, an NPU has to be flexible enough to accelerate a dominant fraction of

the computational kernels in a neural network, to ensure that $f \ll 1$. If these kernels are not homogeneous, multiple “sub-accelerators” may be needed within the NPU. Dimensioning to ensure high utilization for all sub-accelerators, as required to achieve high efficiency, is a difficult balancing act since silicon area is always limited. Several architectures are being explored to address the accelerator utilization challenge [76], and this is an active area of research: the sweet spot between efficiency and flexibility depends not only on workload, but also on technology parameters, such as leakage per unit area, which grows with technology scaling, penalizing under-utilization not only from the area, but also from the power viewpoint.

In terms of the roofline model, flexible architectures incur the typically substantial overhead of control flow optimized for software extensibility. This has the impact of reducing both the compute and memory-accessing rooflines. The benefits of such architectures, on the other hand, become apparent from the utilization achieved across a diverse set of workloads (i.e., workload operating points near the roofline). Since the architectural overheads of software-optimized control flow are typically large, judicious selection of the target workload set is often necessary, making ML accelerator design an evolving and application-driven exercise.

V. CONCLUSION AND OPEN CHALLENGES: HOW TO KEEP PUSHING ML ACCELERATOR PERFORMANCE?

The past ten years have seen dramatic enhancements in ML acceleration, amounting to roughly $1000\times$ increase in throughput and energy efficiency. These have come from two synergistic sources: 1) hardware improvements, at the technological, circuit, and architectural levels and 2) algorithmic improvements, in the form of number formats and model architectures, which have, in turn, been leveraged to their full extent by the integration of hardware supports.

Today, we see that both of these are hitting limits, bound in one case by the fundamental underlying technologies and in the other case by ultimate task-level accuracies. This has placed ML accelerator design squarely in a regime of balancing across critical tradeoffs, spanning peak efficiency, peak throughput, achievable utilization, and workload flexibility. This has made it essential to understand the inter-related tradeoffs, in order to effectively design ML accelerators for the diverse range of system deployments being envisioned. A key tool for reasoning about these tradeoffs has been the roofline model, which offers insight into how to compute performance and memory bandwidth constraints shape accelerator efficiency. As ML workloads diversify, accelerator architectures must be designed to not only push peak performance (raising the rooflines), but also ensure high utilization across a range of workloads (operating near the rooflines).

Looking beyond today, the rich range of research makes a number of emerging technologies, across computation, memory, and networking, as well as emerging algorithmic and model architecture innovations, likely to continue to fuel enhancements in ML acceleration. Indeed, maximally leveraging these within future ML accelerators, at the rapid pace with which they are emerging, also makes it essential to understand the tradeoffs covered in this article.

While a complete survey of emerging technologies is beyond the scope of this article, a few examples can help to illustrate how such technologies may influence the tradeoffs discussed. For instance, chiplets and their associated technologies (hybrid integration, die-to-die interconnects) will provide promising pathways for scaling compute parallelism and memory bandwidth, and as such raise the rooflines. This can be further extended to distributed systems, across many compute and memory nodes, where the next-generation interconnect technologies, such as optical communication. Also leveraging emerging memories can provide greater density and enables scale-up of memories to address capacity requirements with ever-increasing model sizes. Yet again, the challenge is not only to make the roofline as high as possible, but also to operate close to it. Fortunately, such scale out could also raise data reuse opportunities and introduce AI relaxations between nodes and clusters, which may make new and existing memory technologies relevant, in repurposed forms and with optimized interfaces.

In addition to interconnect and memory technologies, new computing technologies will also play a key role. Analog computation, particularly in specific forms of in-memory computing, is already showing direct relevance for the next-generation systems. Its potential is currently being explored on a number of other fronts, including alongside new device and materials innovations.

As mentioned, however, in complement to hardware innovations, maximizing utilization of architectures will remain an important challenge and point of system leverage. This will be achievable by more tightly integrating accelerator performance models with compiler technology. Rapid development cycles in both hardware and software suffer from the current need to micro-code kernels in order to achieve satisfactory utilization. In this context, multi-level intermediate representation (MLIR [79]) can form an effective abstraction framework for the development and integration of tools toward rapid processor emulation and code tuning

In the end, designing hardware for ML will always remain a balancing game: maximizing total system efficiency and throughput, while maintaining sufficient flexibility to be prepared for the continuous stream of innovations in ML algorithms. The danger is that new powerful algorithms are of such different nature that they do not match well on today's processor architectures, preventing their breakthrough. To avoid this hardware lottery [80], we need to find a way to not only streamline the design and programming of novel ML accelerator platforms, but also their fabrication. The recent trend toward using chiplet technology not only for achieving scale-out with good yield, but also toward rapid customization seems promising. Future will tell whether this will allow us to again enable the next wave of ML algorithms.

REFERENCES

- [1] G. Anil. (2023). *Visualizing Size of Large Language Models*. Accessed: Oct. 10, 2024. [Online]. Available: <https://medium.com/@georgeanil/visualizing-size-of-large-language-models-ec576caa5557>
- [2] B. Dally, "Trends in deep learning hardware," NVIDIA, Santa Clara, CA, USA, Tech. Rep., 2024. [Online]. Available: <https://www.youtube.com/watch?v=kLiwnr4L80>
- [3] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Commun. ACM*, vol. 52, no. 4, pp. 65–76, Apr. 2009.
- [4] Y.-H. Chen, J. Emer, and V. Sze, "Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks," in *Proc. ACM/IEEE 43rd Annu. Int. Symp. Comput. Archit. (ISCA)*, Jun. 2016, pp. 367–379.
- [5] H. C. Prashanth and M. Rao, "Roofline performance analysis of DNN architectures on CPU and GPU systems," in *Proc. 25th Int. Symp. Quality Electron. Design (ISQED)*, Apr. 2024, pp. 1–8.
- [6] S. W. Williams, *The Roofline Model*. Oakland, CA, USA: University of California, 2010.
- [7] Z. Yuan et al., "LLM inference unveiled: Survey and roofline model insights," 2024, *arXiv:2402.16363*.
- [8] H. Kwon, P. Chatarasi, M. Pellauer, A. Parashar, V. Sarkar, and T. Krishna, "Understanding reuse, performance, and hardware cost of DNN dataflow: A data-centric approach," in *Proc. 52nd Annu. IEEE/ACM Int. Symp. Microarchit.*, Apr. 2019, pp. 754–768.
- [9] J. W. Choi, D. Bedard, R. Fowler, and R. Vuduc, "A roofline model of energy," in *Proc. IEEE 27th Int. Symp. Parallel Distrib. Process.*, May 2013, pp. 661–672.
- [10] M. Pellauer et al., "Symphony: Orchestrating sparse and dense tensors with hierarchical heterogeneous processing," *ACM Trans. Comput. Syst.*, vol. 41, nos. 1–4, pp. 1–30, Nov. 2023.
- [11] T. P. Morgan. (Oct. 2024). *Lots of Questions on Google's 'Trillium' TPU V6, a Few Answers*. [Online]. Available: <https://www.nextplatform.com/2024/06/10/lots-of-questions-on-googles-trillium-tpu-v6-a-few-answers/>
- [12] B. Moons, R. Uytterhoeven, W. Dehaene, and M. Verhelst, "14.5 envision: A 0.26-to-10TOPS/W subword-parallel dynamic-voltage-accuracy-frequency-scalable convolutional neural network processor in 28 nm FDSOI," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2017, pp. 246–247.
- [13] J.-S. Park et al., "9.5 a 6K-MAC feature-map-sparsity-Aware neural processing unit in 5nm flagship mobile SoC," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 64, Feb. 2021, pp. 152–154.
- [14] E. Talpes et al., "Compute solution for Tesla's full self-driving computer," *IEEE Micro*, vol. 40, no. 2, pp. 25–35, Mar. 2020.
- [15] Y. Jiao et al., "7.2 a 12nm programmable convolution-efficient neural-processing-unit chip achieving 825TOPS," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2020, pp. 136–140.
- [16] L. Gwennap, "Groq rocks neural networks," Tech. Rep., Jan. 2020. [Online]. Available: <https://groq.com/wp-content/uploads/2023/05/GROQ-ROCKS-NEURAL-NETWORKS.pdf>
- [17] A. Agrawal et al., "9.1 A 7nm 4-core AI chip with 25.6TFLOPS hybrid FP8 training, 102.4TOPS INT4 inference and workload-aware throttling," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 64, May 2021, pp. 144–146.
- [18] G. Desoli et al., "16.7 a 40–310TOPS/W SRAM-based all-digital up to 4b in-memory computing multi-tiled NN accelerator in FD-SOI 18nm for deep-learning edge applications," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2023, pp. 260–262.
- [19] J. Zhuang et al., "CHARM: Composing heterogeneous accelerators for matrix multiply on versal ACAP architecture," in *Proc. ACM/SIGDA Int. Symp. Field Program. Gate Arrays*, New York, NY, USA, Feb. 2023, pp. 153–164, doi: [10.1145/3543622.3573210](https://doi.org/10.1145/3543622.3573210).
- [20] H. Liao, J. Tu, J. Xia, and X. Zhou, "DaVinci: A scalable architecture for neural network computing," in *Proc. Hot Chips Symp.*, 2019, pp. 1–44.
- [21] S. Markidis, S. W. D. Chien, E. Laure, I. B. Peng, and J. S. Vetter, "NVIDIA tensor core programmability, performance & precision," in *Proc. IEEE Int. Parallel Distrib. Process. Symp. Workshops (IPDPSW)*, May 2018, pp. 522–531.
- [22] H. Wang, R. Liu, R. Dorrance, D. Dasalukunte, D. Lake, and B. Carlton, "A charge domain SRAM compute-in-memory macro with C-2C ladder-based 8-bit MAC unit in 22-nm FinFET process for edge inference," *IEEE J. Solid-State Circuits*, vol. 58, no. 4, pp. 1037–1050, Apr. 2023.
- [23] I. A. Papistas et al., "A 22 nm, 1540 TOP/s/W, 12.1 TOP/s/mm² in-memory analog matrix-vector-multiplier for DNN acceleration," in *Proc. IEEE Custom Integr. Circuits Conf. (CICC)*, Apr. 2021, pp. 1–2.
- [24] H. Valavi, P. J. Ramadge, E. Nestler, and N. Verma, "A 64-tile 2.4-Mb in-memory-computing CNN accelerator employing charge-domain compute," *IEEE J. Solid-State Circuits*, vol. 54, no. 6, pp. 1789–1799, Jun. 2019.
- [25] B. Dally, "Hardware for deep learning," in *Proc. IEEE Hot Chips 35 Symp. (HCS)*, vol. 35, Aug. 2023, pp. 1–58.

- [26] A. Reuther, P. Michaleas, M. Jones, V. Gadepally, S. Samsi, and J. Kepner, "Lincoln AI computing survey (LAICS) update," in *Proc. IEEE High Perform. Extreme Comput. Conf. (HPEC)*, Sep. 2023, pp. 1–7.
- [27] K. Guo et al. (2018). *Neural Network Accelerator Comparison*. [Online]. Available: <https://nicsefc.ee.tsinghua.edu.cn/project.html>
- [28] L. Bertaccini, G. Paulin, M. Cavalcante, T. Fischer, S. Mach, and L. Benini, "MiniFloats on RISC-V cores: ISA extensions with mixed-precision short dot products," *IEEE Trans. Emerg. Topics Comput.*, vol. 12, no. 4, pp. 1040–1055, Oct. 2024.
- [29] M. Scherer, G. Rutishauser, L. Cavigelli, and L. Benini, "CUTIE: Beyond PetaOp/s/W ternary DNN inference acceleration with better-than-binary energy efficiency," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 41, no. 4, pp. 1020–1033, Apr. 2022.
- [30] B. Moons, D. Bankman, L. Yang, B. Murmann, and M. Verhelst, "Binare-Eye: An always-on energy-accuracy-scalable binary CNN processor with all memory on chip in 28nm CMOS," in *Proc. IEEE Custom Integr. Circuits Conf. (CICC)*, Apr. 2018, pp. 1–4.
- [31] H. Wang et al., "BitNet: Scaling 1-bit transformers for large language models," 2023, *arXiv:2310.11453*.
- [32] A. Nadalini et al., "A 3 TOPS/W RISC-V parallel cluster for inference of fine-grain mixed-precision quantized neural networks," in *Proc. IEEE Comput. Soc. Annu. Symp. VLSI (ISVLSI)*, 2023, pp. 1–6.
- [33] F. Conti et al., "Marsellus: A heterogeneous RISC-V AI-IoT end-node SoC with 2–8 b DNN acceleration and 30%-boost adaptive body biasing," *IEEE J. Solid-State Circuits*, vol. 59, no. 1, pp. 128–142, Jan. 2024.
- [34] B. Darvish Rouhani et al., "Microscaling data formats for deep learning," 2023, *arXiv:2310.10537*.
- [35] A. Tirumala and R. Wong, "NVIDIA Blackwell platform: Advancing generative AI and accelerated computing," in *Proc. IEEE Hot Chips 36 Symp. (HCS)*, Aug. 2024, pp. 1–33.
- [36] A. S. Prasad et al., "Siracusa: A 16 nm heterogeneous RISC-V SoC for extended reality with at-MRAM neural engine," *IEEE J. Solid-State Circuits*, vol. 59, no. 7, pp. 2055–2069, Jul. 2024.
- [37] K. Koul et al., "Onyx: A 12 nm 756 GOPS/W coarse-grained reconfigurable array for accelerating dense and sparse applications," in *Proc. IEEE Symp. VLSI Technol. Circuits (VLSI Technol. Circuits)*, Jun. 2024, pp. 1–2.
- [38] A. Zhou et al., "Learning N: M fine-grained structured sparse neural networks from scratch," 2021, *arXiv:2102.04010*.
- [39] J. Choquette, E. Lee, R. Krashinsky, V. Balan, and B. Khailany, "3.2 the A100 datacenter GPU and ampere architecture," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 64, Feb. 2021, pp. 48–50.
- [40] R. L. Castro, A. Ivanov, D. Andrade, T. Ben-Nun, B. B. Fraguera, and T. Hoefler, "VENOM: A vectorized N: M format for unleashing the power of sparse tensor cores," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, Nov. 2023, pp. 1–14.
- [41] P. Scheffler et al., "Occamy: A 432-core dual-chiplet dual-HBM2E 768-DP-GFLOP/s RISC-V system for 8-to-64-bit dense and sparse computing in 12-nm FinFET," *IEEE J. Solid-State Circuits*, early access, Jan. 30, 2025, doi: [10.1109/JSSC.2025.3529249](https://doi.org/10.1109/JSSC.2025.3529249).
- [42] G. Heo et al., "NeuPIMs: NPU-PIM heterogeneous acceleration for batched LLM inferencing," in *Proc. 29th ACM Int. Conf. Architectural Support Program. Lang. Operating Syst.*, New York, NY, USA, Apr. 2024, pp. 722–737, doi: [10.1145/3620666.3651380](https://doi.org/10.1145/3620666.3651380).
- [43] J. Alsop, S. Aga, M. Ibrahim, M. Islam, A. McCrabb, and N. Jayasena, "Inclusive-PIM: Hardware–software co-design for broad acceleration on commercial PIM architectures," 2023, *arXiv:2309.07984*.
- [44] J. Zhang, Z. Wang, and N. Verma, "In-memory computation of a machine-learning classifier in a standard 6T SRAM array," *IEEE J. Solid-State Circuits*, vol. 52, no. 4, pp. 915–924, Apr. 2017.
- [45] M. Kang, S. K. Gonugondla, M.-S. Keel, and N. R. Shanbhag, "An energy-efficient memory-based high-throughput VLSI architecture for convolutional networks," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, May 2015, pp. 1037–1041.
- [46] X. Guo et al., "Fast, energy-efficient, robust, and reproducible mixed-signal neuromorphic classifier based on embedded NOR flash memory technology," in *IEDM Tech. Dig.*, Dec. 2017, pp. 6.5.1–6.5.4.
- [47] L. Fick, D. Blaauw, D. Sylvester, S. Skrzyniarz, M. Parikh, and D. Fick, "Analog in-memory subthreshold deep neural network accelerator," in *Proc. IEEE Custom Integr. Circuits Conf. (CICC)*, Apr. 2017, pp. 1–4.
- [48] H. Fujiwara et al., "A 5-nm 254-TOPS/W 221-TOPS/mm² fully-digital computing-in-memory macro supporting wide-range dynamic-voltage-frequency scaling and simultaneous MAC and write operations," in *Proc. IEEE Int. Solid-State Circuits Conf.*, Feb. 2022, pp. 1–3.
- [49] Y.-D. Chih et al., "16.4 an 89TOPS/W and 16.3TOPS/mm² all-digital SRAM-based full-precision compute-in memory macro in 22nm for machine-learning edge applications," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 64, Feb. 2021, pp. 252–254.
- [50] P. Deaville, B. Zhang, L.-Y. Chen, and N. Verma, "A maximally row-parallel MRAM in-memory-computing macro addressing readout circuit sensitivity and area," in *Proc. IEEE 51st Eur. Solid-State Device Res. Conf. (ESSDERC)*, Sep. 2021, pp. 75–78.
- [51] H. Jia, H. Valavi, Y. Tang, J. Zhang, and N. Verma, "A programmable heterogeneous microprocessor based on bit-scalable in-memory computing," *IEEE J. Solid-State Circuits*, vol. 55, no. 9, pp. 2609–2621, Sep. 2020.
- [52] S. Jung et al., "A crossbar array of magnetoresistive memory devices for in-memory computing," *Nature*, vol. 601, no. 7892, pp. 211–216, Jan. 2022, doi: [10.1038/s41586-021-04196-6](https://doi.org/10.1038/s41586-021-04196-6).
- [53] N. Verma et al., "In-memory computing: Advances and prospects," *IEEE Solid-State Circuits Mag.*, vol. 11, no. 3, pp. 43–55, Aug. 2019.
- [54] J. Wang et al., "14.2 A compute SRAM with bit-serial integer/floating-point operations for programmable in-memory vector acceleration," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2019, pp. 224–226.
- [55] S. D. Spetalnick et al., "A 40nm 64kb 26.56TOPS/W 2.37Mb/mm²RRAM binary/compute-in-memory macro with 4.23x improvement in density and >75% use of sensing dynamic range," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, vol. 65, Feb. 2022, pp. 1–3.
- [56] S. K. Gonugondla, C. Sakr, H. Dbouk, and N. R. Shanbhag, "Fundamental limits on energy-delay-accuracy of in-memory architectures in inference applications," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 41, no. 10, pp. 3188–3201, Oct. 2022.
- [57] P. A. Hager et al., "11.3 metis AIPU: A 12nm 15TOPS/W 209.6TOPS SoC for Cost- and energy-efficient inference at the edge," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2024, pp. 212–214.
- [58] N. R. Shanbhag and S. K. Roy, "Benchmarking in-memory computing architectures," *IEEE Open J. Solid-State Circuits Soc.*, vol. 2, pp. 288–300, Dec. 2022.
- [59] P. Deaville, B. Zhang, and N. Verma, "A 22nm 128-kb MRAM row/column-parallel in-memory computing macro with memory-resistance boosting and multi-column ADC readout," in *Proc. IEEE Symp. VLSI Technol. Circuits (VLSI Technol. Circuits)*, Sep. 2022, pp. 268–269.
- [60] M. L. Gallo et al., "A 64-core mixed-signal in-memory compute chip based on phase-change memory for deep neural network inference," *Nature Electron.*, vol. 6, no. 9, pp. 680–693, Aug. 2023, doi: [10.1038/s41928-023-01010-1](https://doi.org/10.1038/s41928-023-01010-1).
- [61] C.-C. Chou et al., "An N40 256K×44 embedded RRAM macro with SL-precharge SA and low-voltage current limiter to improve read and write performance," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2018, pp. 478–480.
- [62] D. Shum et al., "CMOS-embedded STT-MRAM arrays in 2x nm nodes for GP-MCU applications," in *Proc. Symp. VLSI Technol.*, Jun. 2017, pp. T208–T209.
- [63] J. Lee, B. Zhang, and N. Verma, "A switched-capacitor SRAM in-memory computing macro with high-precision, high-efficiency differential architecture," in *Proc. IEEE Eur. Solid-State Electron. Res. Conf. (ESSERC)*, Sep. 2024, pp. 357–360.
- [64] H. Jia et al., "Scalable and programmable neural network inference accelerator based on in-memory computing," *IEEE J. Solid-State Circuits*, vol. 57, no. 1, pp. 198–211, Jan. 2022.
- [65] X. Yang et al., "Interstellar: Using halide's scheduling language to analyze DNN accelerators," in *Proc. 25th Int. Conf. Architectural Support Program. Lang. Operating Syst.*, Mar. 2020, pp. 369–383, doi: [10.1145/3373376.3378514](https://doi.org/10.1145/3373376.3378514).
- [66] H. Kwon, P. Chatarasi, V. Sarkar, T. Krishna, M. Pellauer, and A. Parashar, "MAESTRO: A data-centric approach to understand reuse, performance, and hardware cost of DNN mappings," *IEEE Micro*, vol. 40, no. 3, pp. 20–29, May 2020.
- [67] A. Parashar et al., "Timeloop: A systematic approach to DNN accelerator evaluation," in *Proc. IEEE Int. Symp. Perform. Anal. Syst. Softw. (ISPASS)*, Mar. 2019, pp. 304–315.

- [68] L. Mei, P. Houshmand, V. Jain, S. Giraldo, and M. Verhelst, "ZigZag: Enlarging joint architecture-mapping design space exploration for DNN accelerators," *IEEE Trans. Comput.*, vol. 70, no. 8, pp. 1160–1174, Aug. 2021.
- [69] Q. Huang et al., "CoSA: Scheduling by constrained optimization for spatial accelerators," in *Proc. ACM/IEEE 48th Annu. Int. Symp. Comput. Archit. (ISCA)*, Jul. 2021, pp. 554–566.
- [70] K. Hegde, P.-A. Tsai, S. Huang, V. Chandra, A. Parashar, and C. W. Fletcher, "Mind mappings: Enabling efficient algorithm-accelerator mapping space search," in *Proc. 26th ACM Int. Conf. Archit. Support Program. Lang. Oper. Syst.*, New York, NY, USA, 2021, pp. 943–958, doi: [10.1145/3445814.3446762](https://doi.org/10.1145/3445814.3446762).
- [71] S.-C. Kao and T. Krishna, "GAMMA: Automating the HW mapping of DNN models on accelerators via genetic algorithm," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD)*, New York, NY, USA, Nov. 2020, pp. 1–9, doi: [10.1145/3400302.3415639](https://doi.org/10.1145/3400302.3415639).
- [72] A. Symons, L. Mei, S. Coleman, P. Houshmand, S. Karl, and M. Verhelst, "Stream: Design space exploration of layer-fused DNNs on heterogeneous dataflow accelerators," *IEEE Trans. Comput.*, vol. 74, no. 1, pp. 237–249, Jan. 2025.
- [73] D. Abts et al., "The groq software-defined scale-out tensor streaming multiprocessor : From chips-to-systems architectural overview," in *Proc. IEEE Hot Chips 34 Symp. (HCS)*, Aug. 2022, pp. 1–69.
- [74] G. Kappen and T. G. Noll, "Application specific instruction processor based implementation of a GNSS receiver on an FPGA," in *Proc. Design Autom. Test Eur. Conf.*, 2006, pp. 58–63.
- [75] S. Huang, L. Waeijen, and H. Corporaal, "How flexible is your computing system?" *ACM Trans. Embedded Comput. Syst.*, vol. 21, no. 4, pp. 1–41, Jul. 2022.
- [76] S. Ghodrati et al., "Tandem processor: Grappling with emerging operators in neural networks," in *Proc. 29th ACM Int. Conf. Architectural Support Program. Lang. Operating Syst.*, vol. 2, Apr. 2024, pp. 1165–1182.
- [77] M. Cho and D. Brand, "MEC: Memory-efficient convolution for deep neural network," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 815–824.
- [78] S.-C. Kao, H. Kwon, M. Pellauer, A. Parashar, and T. Krishna, "A formalism of DNN accelerator flexibility," *ACM SIGMETRICS Perform. Eval. Rev.*, vol. 50, no. 1, pp. 53–54, Jun. 2022.
- [79] C. Lattner et al., "MLIR: Scaling compiler infrastructure for domain specific computation," in *Proc. IEEE/ACM Int. Symp. Code Gener. Optim. (CGO)*, Feb. 2021, pp. 2–14.
- [80] S. Hooker, "The hardware lottery," *Commun. ACM*, vol. 64, no. 12, pp. 58–65, Dec. 2021.



Marian Verhelst (Fellow, IEEE) received the Ph.D. degree from KU Leuven, Leuven, Belgium, in 2008.

She was a Research Scientist at Intel Labs, from 2008 to 2011. She is currently a Professor at the MICAS Laboratories, KU Leuven, and the Research Director at IMEC, Leuven. She is a scientific advisor to multiple startups. Her research focuses on embedded machine learning, hardware accelerators, and low-power edge processing.

Dr. Verhelst is a member of the Board of ECSA, the Royal Academy of Belgium for Science and Arts, the Young Academy of Belgium, and the STEM Advisory Committee to the Flemish Government, and a TPC Member of ISSCC, ISCA, and ESSERC. She served on the Board of Directors of tinyML. She is a Regular Member of the Nerdland Science Podcast (in Dutch) and the Founding Mother of InnovationLab High School Program, KU Leuven. She received the Laureate Prize of the Royal Academy of Belgium in 2016, the 2021 Intel Outstanding Researcher Award, and the André Mischke YAE Prize for Science and Policy in 2021. She is a Science Communication Enthusiast as an IEEE SSCS Distinguished Lecturer. She is an Associate Editor of IEEE TRANSACTIONS ON VERY LARGE SCALE INTEGRATION (VLSI) SYSTEMS, IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS II: EXPRESS BRIEFS, and IEEE JOURNAL OF SOLID-STATE CIRCUITS.



Luca Benini (Fellow, IEEE) received the Ph.D. degree from Stanford University, Stanford, CA, USA.

He holds the Chair of Digital Circuits and Systems at ETH Zurich, Zurich, Switzerland, and is a Full Professor at the Università di Bologna, Bologna, Italy. His research interests include energy-efficient parallel computing systems, smart sensing micro-systems, and machine learning hardware.

Dr. Benini is a fellow of ACM and a member of the Academia Europaea and Italian Academy of Engineering and Technology. He was a recipient of the 2016 IEEE CAS Mac Van Valkenburg Award, the 2020 EDAA Achievement Award, the 2020 ACM/IEEE A. Richard Newton Award, the 2023 IEEE CS E.J. McCluskey Award, and the 2024 IEEE CS Open Source Hardware Contribution Award.



Naveen Verma (Senior Member, IEEE) received the B.A.Sc. degree in electrical and computer engineering from UBC, Vancouver, BC, Canada, in 2003, and the M.S. and Ph.D. degrees in electrical engineering from MIT, Cambridge, MA, USA, in 2005 and 2009, respectively.

Since July 2009, he has been at Princeton University, Princeton, NJ, USA, where he is currently the Ralph H. and Freda I. Augustine Professor of electrical and computer engineering. He has been involved in a number of technology transfer activities including founding start-up companies. Most recently, he co-founded EnCharge AI, Santa Clara, USA, together with industry leaders in AI computing systems, to commercialize foundational technology developed in his laboratory. His research focuses on advanced sensing and computing systems, exploring how systems for learning, inference, and action planning can be enhanced by algorithms that exploit new sensing and computing technologies. This includes research on large-area flexible sensors, energy-efficient computing architectures and circuits, and machine-learning and statistical-signal-processing algorithms.

Prof. Verma was a recipient of numerous teaching and research awards, including several best paper awards, with his students. He has served as a Distinguished Lecturer for IEEE Solid-State Circuits Society and on a number of conference program committees and advisory groups.

Prof. Verma was a recipient of numerous teaching and research awards, including several best paper awards, with his students. He has served as a Distinguished Lecturer for IEEE Solid-State Circuits Society and on a number of conference program committees and advisory groups.